

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**Проектиране и разработване на уеб базирана
система за електронна търговия с компютърна и
офис техника**

ДИПЛОМНА РАБОТА

Изготвил:

Кирил Вълков, фак. № 121222194

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

ас. маг. инж. Деян Дънешки

София, 2026

СЪДЪРЖАНИЕ

Увод	1
1. Въведение в веб-базирани платформи за онлайн търговия на компютърни части	5
1.1. Преглед на съществуващите решения	5
1.1.1. Обща картина на съвременните решения за електронна търговия	5
1.1.2. Готови платформи: Shopify, WooCommerce и Magento	6
1.1.3. Custom веб приложения и ролята на Spring екосистемата	8
1.1.4. Архитектурни подходи при веб приложенията	8
1.1.5. Позициониране на настоящия проект	9
1.1.6. Изводи от обзорната част	10
1.2. Теоретични основи	10
1.2.1. Клиент-сървърна природа на веб приложението	10
1.2.2. Многослойна архитектура	11
1.2.3. Моделът MVC	11
1.2.4. Dependency Injection и конфигурация на компоненти	13
1.2.5. Домейн модел и обектно-релационно съответствие	13
1.2.6. Валидация на входни данни	14
1.2.7. Управление на потребителска идентичност и защита на паролите	14
1.2.8. Сървърно рендирани шаблони и локализация	15
1.2.9. REST интерфейс и представяне на данни	15
1.2.10. Обобщение на теоретичната рамка	16
2. Проектиране на веб-базираната платформа	17
2.1. Анализ на изискванията и потребителските сценарии	17
2.1.1. Основни участници в системата	17
2.1.2. Основни потребителски цели	18
2.1.3. Функционални изисквания	19
2.1.4. Нефункционални изисквания	22
2.1.5. Ключови потребителски сценарии	23
2.1.6. Ограничения, произтичащи от текущата постановка	24

2.1.7.	Изводи	24
2.2.	Архитектура на системата	24
2.2.1.	Обща архитектурна постановка	24
2.2.2.	Пакетна организация на сорс кода	26
2.2.3.	Входна точка и стартиране на приложението	27
2.2.4.	Конфигурационен слой	27
2.2.5.	Уеб слой: контролери и маршрути	27
2.2.6.	Service слой: сърце на бизнес логиката	28
2.2.7.	Persistence слой и repository компоненти	29
2.2.8.	Модели за трансфер на данни между слоевете	29
2.2.9.	Представящ слой: шаблони и статични ресурси	30
2.2.10.	Архитектурен поток на ключови операции	30
2.2.11.	Силни страни на архитектурното решение	31
2.2.12.	Архитектурни компромиси и ограничения	31
2.2.13.	Архитектурно обобщение	32
2.3.	Домейн модел и модел на данните	32
2.3.1.	Основни домейн понятия	32
2.3.2.	Entity моделът в JPA слоя	33
2.3.3.	Атрибутен анализ на основните entity класове	35
2.3.4.	Изброим тип за категориите	36
2.3.5.	Binding модели и входна валидация	36
2.3.6.	Service и view модели	37
2.3.7.	Преобразувания между моделите	38
2.3.8.	Съответствие между домейн модел и потребителски сценарии . .	38
2.3.9.	Критични наблюдения върху модела	38
2.3.10.	Обобщение на проектирането	39
3.	Програмна реализация	40
3.1.	Реализация на back-end логиката	40
3.1.1.	Конфигурация на основните инфраструктурни компоненти . . .	40
3.1.2.	Потребителски модул	41
3.1.3.	Модул за фирми	43
3.1.4.	Модул за продукти и каталог	43

3.1.5.	Количка и финализиране на поръчка	45
3.1.6.	REST слой	47
3.1.7.	Роля на repository интерфейсите в реализацията	47
3.1.8.	Наблюдавани силни страни на back-end реализацията	47
3.1.9.	Ограничения и технически рискове в back-end слоя	48
3.2.	Представящ слой и потребителско взаимодействие	48
3.2.1.	Организация на шаблоните	49
3.2.2.	Навигация и общи интерфейсни елементи	51
3.2.3.	Thymeleaf като механизъм за свързване на данни и интерфейс	51
3.2.4.	Форми и обработка на потребителски вход	52
3.2.5.	Каталог и продуктови детайли	52
3.2.6.	Количка и checkout екран	53
3.2.7.	Профил и история на поръчките	53
3.2.8.	Локализация и двуезичен интерфейс	54
3.2.9.	Използване на външни библиотеки и клиентски ресурси	54
3.2.10.	UX силни страни	54
3.2.11.	Ограничения на представящия слой	55
3.3.	Структура на проекта, конфигурация и експлоатационна среда	55
3.3.1.	Обща организация на проекта	56
3.3.2.	Описание на зависимостите в pom.xml	57
3.3.3.	Версия на платформата и езика	59
3.3.4.	Конфигурация на приложението	59
3.3.5.	Конфигурация на електронната поща	59
3.3.6.	Build процес и Maven жизнен цикъл	60
3.3.7.	Съдържание и роля на target	60
3.3.8.	Практическа среда за изпълнение	61
3.3.9.	Операционни рискове и възможности за подобрене	61
3.3.10.	Обобщение на програмната реализация	62
4.	Ръководство за използване	63
4.1.	Потребителски сценарии	63
4.1.1.	Сценарий 1: Начална страница и избор на език	63
4.1.2.	Сценарий 2: Регистрация на нов потребител	64

4.1.3.	Сценарий 3: Потвърждение на акаунт и вход	65
4.1.4.	Сценарий 4: Профил на потребителя и списък с фирми	66
4.1.5.	Сценарий 5: Добавяне на фирма	66
4.1.6.	Сценарий 6: Добавяне на продукт	67
4.1.7.	Сценарий 7: Каталог и филтриране по категория и цена	68
4.1.8.	Сценарий 8: Детайли на продукт и добавяне в количка	69
4.1.9.	Сценарий 9: Количка и финализиране на поръчка	70
4.1.10.	Сценарий 10: Екран за успешна поръчка и история	70
4.1.11.	Сценарий 11: Демонстрация на локализацията	71
4.1.12.	Препоръки за създаване на screenshots	72
4.2.	Верификация на функционалността	72
4.2.1.	Подход към верификацията	73
4.2.2.	Матрица за функционална верификация	74
Заклучение		76
Използвана литература		81

УВОД

Електронната търговия е сред най-динамично развиващите се области на съвременните информационни системи. За разлика от ранните уеб каталози, които изпълняват предимно представителни функции, днешните онлайн магазини представляват комплексни приложения, в които се преплитат процеси по идентификация на потребители, представяне на продукти, управление на наличности, обработка на поръчки, интеграция с външни услуги и устойчиво съхранение на бизнес данни. Дигитализацията на търговските процеси продължава да измества традиционните форми на продажба към уеб и мобилни платформи, а потребителите очакват постоянен достъп до каталожна информация, лесно сравнение между продукти, бърз процес на поръчка и проследимост на взаимодействието със системата. Това поставя високи изисквания пред разработчиците на уеб приложения както по отношение на функционалното покритие, така и по отношение на архитектурната подреденост на решението.

Конкретната предметна област на настоящата работа е насочена към продажба на компютърна и офис техника. Този избор не е случаен – подобни продукти са масово търсени както от индивидуални потребители, така и от фирмени клиенти, и предполагат ясно структуриране по категории, проследяване на наличности, асоцииране с доставчици или фирми и поддържане на сравнително богата каталожна информация. Наличието на продуктови категории като видеокарти, процесори, слушалки, принтери, офис столове, скенери, рутери, мишки и клавиатури прави системата достатъчно разнообразна, за да демонстрира работа с категоризация, филтриране, наличие и потребителски избор. Така предметната област е едновременно достатъчно богата, за да наложи реални архитектурни решения, и достатъчно ограничена, за да може да бъде обхваната в рамките на дипломна разработка.

При изграждането на онлайн магазин възникват няколко взаимосвързани проблема. Системата трябва да предлага интуитивен потребителски интерфейс за разглеждане и избор на продукти, но едновременно с това да поддържа надеждна вътрешна организация на данните. Потребителите трябва да могат да се регистрират, да влизат в профилите си, да добавят продукти в количка и да финализират поръчки. Същевременно приложението трябва да управлява информация за фирми, категории и наличности, като запазва консистентност между визуалното представяне и реалното състояние на данните в

базата. Особено предизвикателство е нуждата от ясно архитектурно разделение: ако цялата логика бъде концентрирана в контролерите или шаблоните, приложението бързо става трудно за поддръжка и разширяване. Затова е необходимо решение, при което представянето, бизнес логиката и достъпът до данни са отделени в самостоятелни слоеве. Допълнително усложнение възниква от необходимостта за обработка на потребителски вход, валидиране на данни, защита на паролите и интеграция с електронна поща за потвърждение на регистрацията.

Основната цел на дипломната работа е да се проектира и разработи уеб базирана система за електронна търговия с компютърна и офис техника, която да поддържа основните процеси по регистрация на потребители, управление на фирми и продукти, разглеждане на каталог, работа с количка, финализиране на поръчка и проследяване на история на покупки. Постигането на тази цел предполага не само създаване на работещ код, но и обосноваване на използваните архитектурни и технологични решения, така че документът да показва не само какво прави приложението, но и защо е избран именно този подход. За реализирането ѝ са формулирани основните задачи на разработката: анализ на предметната област и потребителските сценарии, избор на подходящ технологичен стек за сървърно рендирано Java уеб приложение, проектиране на многослойна архитектура, моделиране на домейн обектите, реализация на регистрация и сесийно управление с потвърждение чрез електронна поща, изграждане на каталог с категоризация и филтриране, реализиране на процеса по оформяне на поръчка и архивиране на историята, базова локализация на интерфейса, примерен REST интерфейс, анализ на структурата на проекта и оценка на силните страни и ограниченията на системата.

Разработеният проект е реализиран като Java уеб приложение върху Spring екосистемата с използване на Java 11 и Spring Boot 2.6.2. Обработката на уеб заявките и връзката между бизнес логиката и потребителския интерфейс са реализирани чрез Spring MVC, а визуалният слой е изграден със сървърно рендирани Thymeleaf шаблони. За достъп до данните се използват Spring Data JPA и MySQL, като моделът на приложението е представен чрез entity класове, repository интерфейси и service слой. Допълнително са приложени Hibernate Validator за входна валидация, ModelMapper за преобразуване между различни моделни представяния и Spring Security Crypto за защитено хеширане на пароли чрез Pbkdf2PasswordEncoder. Изпращането на имейл за потвърждение на регистрацията

е реализирано посредством инфраструктурата на Spring за работа с електронна поща. Този избор позволява използване на добре утвърдени индустриални практики при изграждане на многослойни информационни системи и едновременно с това оставя проекта в разумни инфраструктурни граници, подходящи за учебно-приложна реализация.

Обхватът на разработката включва back-end реализация на основните бизнес процеси, уеб интерфейс чрез HTML шаблони и статични ресурси, персистентност на данни чрез MySQL и JPA модел, локална конфигурация за връзка с база данни и SMTP сървър, както и стандартна Maven организация с директории `src` за изходен код и `target` за генерирани артефакти. Извън обхвата на настоящата работа съзнателно остават няколко характерни за производствените системи елементи: интеграция с реален платежен оператор, роля-базирана система за авторизация чрез Spring Security, контейнеризация с Docker, централизирано управление на тайни и разширена система от автоматизирани тестове. Тези аспекти са важни и са оставени като естествено поле за бъдещо развитие, тъй като основната цел тук е изграждането на функционална и добре структурирана учебно-приложна система.

Подходът към разработката съчетава елементи от анализ, проектиране, имплементация и последваща оценка. Първо е разгледана предметната област и са извлечени основните сценарии, които системата следва да покрива. След това е избран технологичен стек, достатъчно зрял и добре документиран за изграждане на многослойно Java уеб приложение. Следват моделиране на ключовите домейн обекти и дефиниране на връзките между тях, последвани от реализация, при която отговорностите са разпределени в отделни пакети и слоеве – контролери за HTTP заявките, услуги за бизнес логиката, repository интерфейси за достъп до базата данни и шаблони за визуализация. Допълнително се използват binding, service и view модели за ограничаване на зависимостите между слоевете. Оценката на решението е предимно аналитична и сценарийна, като стъпва върху изследване на реалния код, маршрути, шаблони, конфигурация и данни и формулира както постигнатите резултати, така и ограниченията на текущата имплементация.

Практическата стойност на проекта се изразява в няколко направления. Създадена е реално стартираща основа за онлайн магазин, която може да бъде използвана като демонстрационен проект, учебна база или отправна точка за по-нататъшно надграждане. Системата демонстрира как могат да бъдат комбинирани ключови Spring технологии

в рамките на един последователен архитектурен модел. Документацията осигурява систематично описание на решенията в проекта, което улеснява поддръжката, разбирането и бъдещото развитие. В образователен план проектът показва как абстрактни понятия като MVC, dependency injection, ORM, валидация и транзакционност се материализират в конкретни класове, маршрути, шаблони и конфигурации, превръщайки системата в удобен мост между теорията и практиката.

Изложението на дипломната работа е организирано в четири основни глави. Първата глава въвежда контекста на уеб-базираните платформи за онлайн търговия на компютърни части, прави преглед на съществуващите решения и въвежда теоретичните основи, върху които стъпва архитектурата. Втората глава представя проектирането на уеб-базираната платформа чрез анализ на изискванията, архитектурната организация и домейн модела. Третата глава е посветена на програмната реализация на back-end логиката, представящия слой и операционната среда на приложението. Четвъртата глава представя ръководство за използване на системата чрез поредица потребителски сценарии и верификация на реализираната функционалност. Заключението обобщава постигнатото, прави оценка на качествените характеристики на решението, идентифицира ограниченията му и очертава насоки за бъдещо развитие.

1. ВЪВЕДЕНИЕ В УЕБ-БАЗИРАНИ ПЛАТФОРМИ ЗА ОНЛАЙН ТЪРГОВИЯ НА КОМПЮТЪРНИ ЧАСТИ

Настоящата глава поставя разработката в контекста на съществуващите подходи и теоретичните основи на уеб-базираните платформи за електронна търговия. Първо се прави преглед на типичните решения в областта – от готови SaaS платформи до custom приложения върху общи уеб рамки – и се обосновава изборът на самостоятелна разработка. След това се въвеждат основните теоретични понятия, върху които стъпва архитектурата на системата: клиент–сървърно взаимодействие, многослойна организация, MVC модел, обектно-релационно моделиране, валидация на данни и сесийно управление.

1.1. Преглед на съществуващите решения

За да бъде правилно оценена една софтуерна разработка, тя следва да бъде позиционирана спрямо вече съществуващите решения в областта. Това е особено важно при системите за електронна търговия, тъй като пазарът предлага както готови платформи с богата функционалност, така и множество технологични стекове за изграждане на custom приложения. Настоящият проект не възниква във вакуум; той е реализиран в контекста на вече наложени модели за организация на каталог, поръчка, потребителски профил и административни функции. Целта на настоящия раздел е да очертае този контекст и да покаже с какво разглежданата система се доближава до утвърдените практики и в какво съзнателно се различава.

1.1.1. Обща картина на съвременните решения за електронна търговия

Съвременните системи за електронна търговия могат условно да се разделят на три основни групи. Първата група включва напълно управлявани SaaS платформи, при които доставчикът поема голяма част от инфраструктурните и административните задачи. Втората група включва self-hosted системи с готова функционалност и възможност за разширяване чрез плъгини и теми. Третата група обхваща custom решения, разработени върху общопредназначени уеб рамки, при които логиката, структурата на данните и интерфейсът се изграждат специално за конкретния проект.

Изборът между тези подходи зависи от множество фактори: степен на

необходима гъвкавост, бюджет, срокове, изисквания към интеграции, организационен контекст и налични технически компетентности. Готовите платформи предлагат бърз старт и богат набор от стандартни функции, но ограничават архитектурния контрол. Custom разработката изисква повече проектантски и програмни усилия, но предоставя възможност системата да бъде моделирана точно според целите на екипа или учебната задача. В този смисъл настоящият проект попада в третата група и следва да бъде разглеждан като контролирана инженерна реализация, а не като конфигуриране на готов продукт.

1.1.2. Готови платформи: Shopify, WooCommerce и Magento

Една от най-разпознаваемите SaaS платформи за електронна търговия е Shopify [1]. Нейното основно предимство е, че предлага цялостна инфраструктура, готов административен панел, управление на продукти, теми, плащания и интеграции с външни услуги. За бизнеси, които търсят бързо излизане на пазара и не се нуждаят от ниско ниво на контрол над архитектурата, Shopify е разумен избор. От гледна точка на дипломна разработка обаче този тип платформа е по-слабо подходяща, защото голяма част от инженерните решения са вече предварително взети от доставчика, а възможностите за задълбочен архитектурен анализ са ограничени.

WooCommerce [2] представлява различен подход. То е разширение за WordPress, което превръща система за управление на съдържание в електронен магазин. Предимството му е в лесната инсталация, голямата екосистема от теми и плъгини и ниската бариера за въвеждане. Подходът е особено полезен, когато вече съществува WordPress сайт и електронният магазин трябва да бъде интегриран към него. Недостатъкът е, че архитектурата на приложението остава силно зависима от структурата на WordPress и от начина, по който плъгините взаимодействат помежду си. Това ограничава чистотата на модела и прозрачността на вътрешната бизнес логика.

Magento Open Source [3] е пример за мощна self-hosted платформа, ориентирана към по-сложни сценарии, по-големи каталози и по-високи изисквания към конфигурация и разширяване. Тя предлага богат набор от модули, административни възможности и интеграционни точки. В същото време сложността ѝ е значително по-висока от тази на WooCommerce или SaaS решенията. За малка или учебна разработка такава платформа често е прекалено тежка, тъй като значителна част от усилието се пренасочва към усвояване на рамката и нейните вътрешни механизми, вместо към ясно проектно

моделиране на конкретното приложение.

Съпоставката между тези платформи показва, че готовите решения са силни, когато бизнесът има нужда от бърза operational готовност. Те обаче не са оптимални, когато основната цел е демонстрация и анализ на фундаментални принципи от софтуерното инженерство. Именно поради това в настоящия проект е предпочетена самостоятелна разработка върху обща уеб рамка.

Таблица 1. Съпоставка между типични подходи за изграждане на електронен магазин

Подход	Предимства	Ограничения	Подходящост за дипломна разработка
Shopify	Бърз старт, хоствана инфраструктура, готови интеграции	Ограничен архитектурен контрол, зависимост от доставчик	Ниска, защото инженерните решения са силно абстрахирани
WooCommerce	Лесно внедряване, богата екосистема, ниска бариера за вход	Силна зависимост от WordPress, натрупване на плъгини	Средна, ако фокусът е върху конфигурация, не върху архитектура
Magento	Богата функционалност, разширяемост, зрял търговски модел	Висока сложност и тежка среда за внедряване	Средна към ниска при ограничен времеви ресурс
Custom Spring приложение	Пълен контрол върху модел, код и архитектура	По-голяма първоначална разработка	Висока, защото позволява анализ на всички слоеве

Таблица 1 показва, че custom Spring приложение не е най-бързият път към готова търговска платформа, но е най-подходящият избор, когато целта е демонстрация на архитектурна последователност, моделиране на предметна област и проследимост на решенията от потребителския интерфейс до базата данни.

1.1.3. Custom уеб приложения и ролята на Spring екосистемата

В Java екосистемата Spring Boot е сред най-често използваните средства за бързо изграждане на самостоятелни уеб приложения [4]. Неговото основно предимство се състои в стандартизирания модел за конфигурация, автоматична инициализация на инфраструктурата и лесна интеграция между различни модули като MVC, сигурност, достъп до данни, шаблони и външни услуги. Това го прави подходяща основа за приложения, които трябва да имат ясен архитектурен гръбнак, но да не се обременяват от прекомерна ръчна конфигурация.

Spring MVC [5] е особено важен в контекста на настоящата разработка, защото предлага естествен модел за маршрутизация на HTTP заявки, binding на параметри, интеграция с валидация и лесна връзка с шаблонни енджини като Thymeleaf [6]. По този начин всеки URL маршрут може ясно да бъде свързан с контролер, бизнес операция и визуализация. Това е съществено предимство при дипломна разработка, защото позволява по-ясно описание на потоците в системата.

Spring Data JPA [7] също играе важна роля. Вместо значителна част от времето да бъде изразходвана за ръчно писане на SQL слой и трансформации между резултатни множества и обекти, JPA моделът позволява домейн обектите да бъдат описани като entity класове с релации, а repository интерфейсите да делегират част от рутинната инфраструктурна логика на рамката. Това не премахва нуждата от добро моделиране, но ускорява реализацията на устойчив слой за съхранение на данни.

За малки и средни приложения изборът на Spring Boot носи и друго предимство: позволява решението да остане монолитно в архитектурен смисъл, но вътрешно добре разделено по модули. Това е особено подходящо за учебен проект, при който е по-важно вътрешната организация да бъде прозрачна и последователна, отколкото приложението да бъде преждевременно разделяно в множество услуги.

1.1.4. Архитектурни подходи при уеб приложенията

Освен между различни платформи, изборът трябва да бъде направен и между различни архитектурни парадигми на самото уеб приложение. В най-общ план могат да бъдат разграничени два доминиращи модела. Първият е сървърно рендираното приложение, при което HTML се генерира на сървъра и се изпраща към клиента като готова страница. Вторият е клиентски ориентираният модел, при който front-end приложението

е самостоятелно, а сървърът предоставя основно REST или GraphQL API.

Сървърно рендираният модел има няколко съществени предимства за разглеждания проект. Той намалява общата архитектурна сложност, тъй като представящият слой и бизнес логиката остават в една и съща технологична среда. Освен това позволява бърза реализация на форми, валидации и навигационни потоци, без да е необходимо изграждането на отделно front-end приложение с независим процес на компилация и разгръщане. За дипломен проект, чиято основна цел е да покаже цялостен back-end и UI процес с ограничен технологичен риск, това е рационален избор.

Клиентски ориентираният модел, типичен за приложения, базирани на React, Angular или Vue, предоставя по-голяма динамичност на интерфейса и по-естествена интеграция с мобилни и външни клиенти. Той обаче изисква отделна front-end архитектура, по-ясно API контракуване, повече внимание към клиентското състояние и често по-сложен модел за сигурност и автентикация. В контекста на текущата задача това би довело до разширяване на технологичния обхват за сметка на фокуса върху систематичната реализация на основните бизнес процеси.

Настоящият проект избира хибриден, но доминиращо сървърно рендиран подход. Основната функционалност се визуализира чрез Thymeleaf, докато отделна REST крайна точка е използвана за демонстрация на програмно достъпна бизнес информация. По този начин системата запазва простота, но същевременно показва възможност за комбиниране на HTML и JSON интерфейси.

1.1.5. Позициониране на настоящия проект

Разработваната система следва да бъде разбрана като custom монолитно приложение, ориентирано към учебно-приложна стойност. То не се конкурира пряко с пълнофункционални търговски платформи, а по-скоро демонстрира как чрез добре подбрани технологии могат да бъдат реализирани основните механизми на електронен магазин в прозрачна и разбираема архитектурна форма.

Проектът заема междинна позиция между два екстрема. От едната страна са платформите тип „всичко готово“, които скриват значителна част от инженерния процес. От другата страна са големите enterprise решения с микросервизна или силно модулна архитектура, чиято сложност е непропорционална на целите на дипломната задача. Настоящата система избира балансиран път: достатъчно реалистична функционалност, реална база данни, реални HTML шаблони, реална e-mail интеграция и реална структура

на сорс кода, но без прекомерни инфраструктурни усложнения.

Тази позиция я прави особено подходяща за академичен анализ. В нея могат ясно да бъдат проследени домейн обектите, заявките към системата, маршрутизацията, ролята на валидацията, управлението на сесията, трансформациите между модели и мястото на конфигурацията. Именно тази проследимост е едно от основните предимства на избрания подход.

1.1.6. Изводи от обзорната част

Прегледът на съществуващите решения показва, че в областта на електронната търговия няма универсално най-добър подход; подходящият избор зависи от целите на конкретния проект. Ако основната задача е бързо въвеждане в експлоатация, SaaS и плъгин-базираните решения имат ясни предимства. Ако обаче водеща е необходимостта от архитектурна прозрачност, контрол върху бизнес логиката и възможност за детайлно инженерно описание, custom уеб приложение върху Spring екосистемата е по-подходящ избор.

Именно в този смисъл настоящата дипломна разработка е обоснована: тя не се стреми да възпроизведе в пълнота зрелите търговски платформи, а да изгради представителен, работещ и аналитично обясним модел на онлайн магазин.

1.2. Теоретични основи

За да бъде разбрана и аргументирана архитектурата на разглежданата система, е необходимо да бъдат въведени няколко основни теоретични понятия. Макар проектът да има подчертано практическа насоченост, неговата реализация стъпва върху утвърдени концепции от областта на уеб инженерството и софтуерната архитектура: клиент–сървърно взаимодействие, многослойна организация, MVC модел, обектно-релационно моделиране, управление на сесия и валидация на данни. Този раздел не цели изчерпателен теоретичен обзор, а по-скоро въвежда именно онези идеи, които са пряко необходими за разбиране на последващото описание на системата.

1.2.1. Клиент–сървърна природа на уеб приложението

В основата на всяка уеб система стои клиент–сървърният модел. Клиентът, най-често уеб браузър, инициира HTTP заявка към сървъра. Сървърът интерпретира

заявката, прилага бизнес логика, извлича или записва данни и връща отговор, който може да бъде HTML документ, пренасочване, файл или структуриран формат като JSON. Тази обща схема е в сила и за разглежданата система: браузърът на потребителя изпраща заявки към контролерите на приложението, а те в зависимост от маршрута връщат страници, извършват пренасочване или генерират REST отговор.

От инженерна гледна точка важна е идеята, че протоколът HTTP е по своята природа безсъстояниен. Това означава, че отделните заявки са независими една от друга, освен ако приложението не въведе собствен механизъм за проследяване на потребителския контекст. В настоящия проект тази роля се поема от `HttpSession`, където се съхранява електронната поща на текущо удостоверения потребител. Така системата постига достатъчно прост модел на сесийно състояние, подходящ за учебно приложение, без да въвежда по-сложни механизми като токени, филтри и централизирана сигурност.

1.2.2. Многослойна архитектура

Многослойната архитектура е един от най-устойчивите организационни модели в enterprise разработката [8, 9]. Нейната идея е различните отговорности в системата да бъдат разделени на слоеве с относително ясни граници. В типичния случай могат да бъдат разграничени визуален слой, приложен слой за управление на заявки и потоци, бизнес слой за домейн логика и слой за достъп до данни.

Предимството на този модел е, че промени в един слой могат да бъдат извършвани с по-малко странични ефекти върху останалите. Например промяна в HTML шаблон не би следвало да налага промяна в entity класовете, а промяна в начина на съхранение на данни не би следвало да засяга непосредствено логиката на формите. Разбира се, в реални системи винаги има определено взаимодействие между слоевете, но многослойният модел създава дисциплина, която намалява вероятността от силно преплитане на отговорности.

В настоящия проект многослойността е реализирана чрез контролери, услуги, repository интерфейси и шаблони. Този подход е особено подходящ за образователна разработка, защото позволява лесно да се проследи движението на данните от потребителската форма към persistence слоя и обратно.

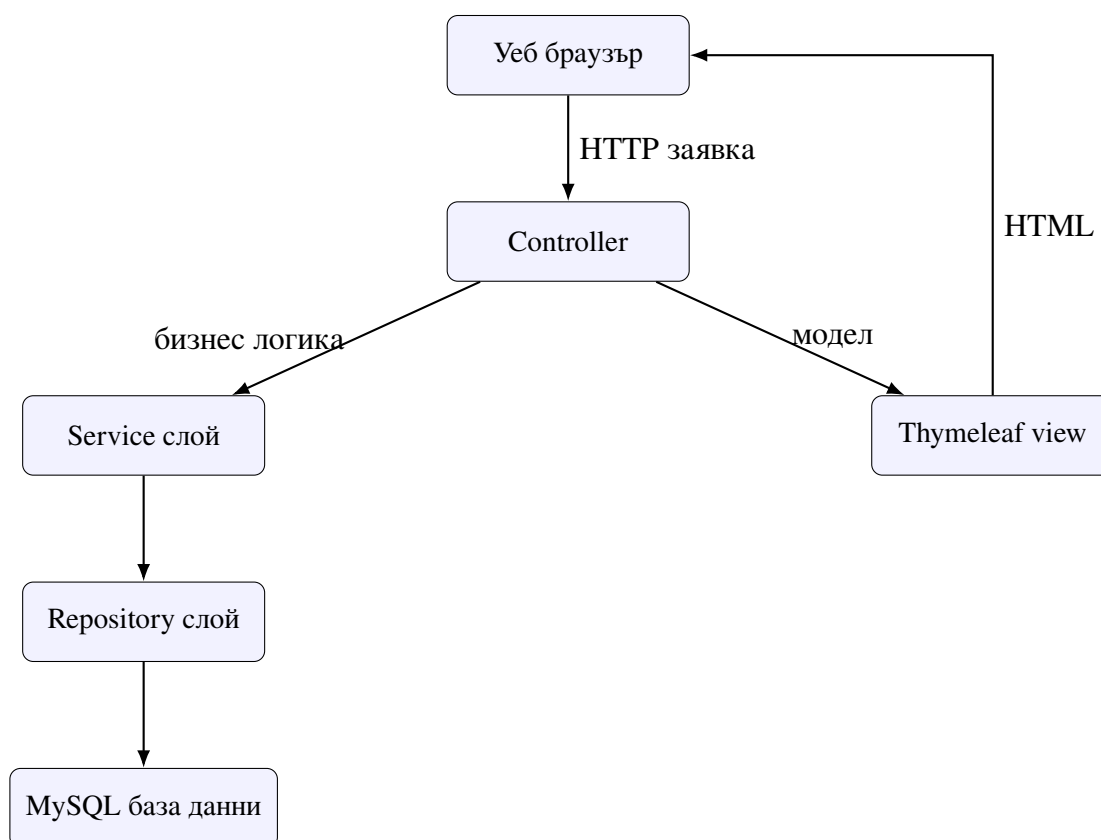
1.2.3. Моделът MVC

Model–View–Controller е класически архитектурен шаблон за уеб приложения и графични системи [5]. В контекста на Spring MVC той се проявява по следния начин:

- **Model** представя данните, които се подават към визуализацията;
- **View** е шаблонът, който визуализира тези данни пред потребителя;
- **Controller** обработва входящата заявка и избира коя view компонента да бъде върната.

Важно е да се подчертае, че в Spring MVC терминът „модел“ не съвпада напълно с домейн модела на приложението. В конкретната реализация домейн обектите са entity класовете, а обектът Model в контролерите представлява контейнер за атрибути, които ще бъдат достъпни от Thymeleaf шаблона. Това разграничение е съществено, защото показва, че MVC не е заместител на домейн модела, а механизъм за организиране на взаимодействието между потребителската заявка и визуализирания резултат.

В разглежданата система MVC моделът е видим почти във всеки маршрут. Контролерът получава заявка за регистрация, логин, каталог, количка или профил; извиква съответна услуга; поставя резултата в модела; и връща име на шаблон. Тази последователност е една от причините проектът да бъде добър пример за ясна учебна архитектура.



Фигура 1. Обобщен поток на обработка при сървърно рендирана заявка

Фигура 1 илюстрира основния път на заявката в системата. Макар той да изглежда линеен, зад всяка стъпка стои отделен набор от отговорности: контролерите управляват маршрутизацията, услугите формулират правилата на домейна, repository компонентите делегират работата с базата данни, а шаблоните осигуряват потребителската визуализация.

1.2.4. Dependency Injection и конфигурация на компоненти

Една от основните идеи в Spring е *dependency injection*, тоест доставяне на необходимите зависимости от външен контейнер, вместо създаването им на ръка във всеки клас. Практически това позволява класовете да бъдат по-слабо свързани, по-лесно тествани и по-ясно конфигурирани [4]. Когато дадена услуга зависи от repository интерфейс, model mapper или password encoder, тя ги получава чрез конструктора си, вместо сама да ги инстанцира.

В текущия проект това е реализирано последователно: контролерите приемат необходимите услуги и инфраструктурни компоненти чрез конструкторна инжекция, а конфигурационният клас дефинира bean инстанции за `ModelMapper` и `PasswordEncoder`. Това решение е важно не само за чистотата на кода, но и за архитектурната проследимост: когато се разглежда даден клас, веднага се вижда от кои други компоненти зависи и каква роля изпълнява.

1.2.5. Домейн модел и обектно-релационно съответствие

Информационните системи често трябва да представят едни и същи понятия в два различни свята: обектния свят на програмния език и релационния свят на базата данни. Обектно-релационното съответствие се стреми да преодолее тази разлика чрез механизъм, при който класовете и връзките между тях се свързват с таблици, колони и външни ключове [7]. В Java екосистемата това обикновено се реализира чрез JPA анотации върху entity класове.

В разглежданата система класове като `User`, `Company`, `Product`, `ShoppingCartEntity` и `HistoryEntity` са маркирани като entity обекти и описват основните релации в домейна. Така се постига сравнително естествена връзка между предметната област и persistence слоя. Например един потребител има множество фирми, количка и история на поръчки; продуктът принадлежи към фирма; а записът в количката свързва конкретен продукт с конкретен потребител и желано количество.

Тук е важно да се направи разграничение между *домейн модел* и *модели за пренос на данни*. В проекта не всички данни се обменят чрез entity обектите. За някои операции са въведени binding модели за входни форми, service модели за вътрешен трансфер между слоеве и view модели за изход към интерфейса. Това е добър архитектурен избор, тъй като ограничава директното изтичане на persistence структурата към потребителския интерфейс.

1.2.6. Валидация на входни данни

Въвеждането на данни от потребител е една от най-честите причини за логически грешки, нарушаване на ограничения в базата или несъответствие между очакваните и действителните стойности. Поради това валидацията на входните данни е фундаментална характеристика на всяка надеждна информационна система. В Spring приложенията това често се реализира чрез Java Bean Validation анотации и обработка на `BindingResult` [10].

В конкретния проект ограниченията са дефинирани основно в binding моделите. За полета като име, адрес, количество, цена и други са зададени правила от типа минимална и максимална дължина, непразна стойност или положително число. Този подход има няколко предимства. Първо, позволява формалното описание на допустимите входни стойности да бъде концентрирано на едно място. Второ, дава възможност контролерът да реагира систематично при невалиден вход, като върне потребителя към съответната форма с индикация за грешка. Трето, намалява риска в service слоя да достигат невалидни обекти.

Въпреки това е важно да се отбележи, че валидацията в текущия проект е частична. Например някои binding модели не съдържат богато множество от ограничения, а част от логическите проверки се извършват на по-късен етап в контролерите или услугите. Това е приемливо за учебен проект, но при по-зряла система би било полезно валидационните правила да бъдат още по-последователни и формализирани.

1.2.7. Управление на потребителска идентичност и защита на паролите

Системите за електронна търговия оперират с чувствителни данни и поради това дори базовият модел на автентикация трябва да бъде проектиран разумно. Един от най-важните принципи е паролите никога да не се съхраняват в явен вид, а да бъдат обработвани чрез устойчив механизъм за хеширане [11, 12]. В текущия проект тази роля

се изпълнява от `Pbkdf2PasswordEncoder`, който осигурява по-добра защита от директно съхраняване или използване на бързи, неподходящи за пароли хеш функции.

Моделът на удостоверяване в проекта е сравнително опростен. При успешен вход електронната поща на потребителя се записва в `HttpSession`. От тази стойност впоследствие се определя дали дадена заявка е позволена и кой е текущият потребител. Този подход е лесен за разбиране и достатъчен за малка учебна система, но има ограничена изразителност спрямо по-пълноценни решения със сигурностен контекст, роли, филтри и декларативни правила за достъп. По-късно в документа това ще бъде разгледано като естествена възможност за бъдещо надграждане.

1.2.8. Сървърно рендирани шаблони и локализация

Thymeleaf е шаблонен механизъм за сървърно рендиране, който интегрира естествено HTML маркировка и изрази за работа с данни от модела [6]. Неговото предимство е, че прави HTML шаблоните сравнително четими и при липса на изпълнение, а в същото време позволява условна визуализация, итериране по колекции, двупосочно свързване на форми и използване на локализирани съобщения.

В настоящия проект Thymeleaf се използва като основен механизъм за изграждане на потребителския интерфейс. Формите за регистрация, вход, добавяне на компания, добавяне на продукт, профил, каталог, количка и история са реализирани именно по този начин. Допълнителен теоретичен аспект тук е локализацията. Чрез `message bundles` и интерцептор за смяна на езика системата демонстрира базов модел за поддръжка на повече от един език, като конкретно предлага английски и български вариант на част от интерфейсите текстове.

1.2.9. REST интерфейс и представяне на данни

Макар приложението да е доминиращо сървърно рендирано, в него е наличен и примерен REST интерфейс, връщащ информация за максималната цена на продукт. Теоретичното значение на тази част е, че показва как една и съща бизнес логика може да бъде използвана през различни форми на представяне. Вместо HTML, при REST сценария резултатът се сериализира до JSON и може да бъде използван от JavaScript код в браузъра или от външен клиент. Подобен стил на взаимодействие е в съзвучие с идеите на уеб архитектурата и ресурсно ориентираното представяне на данни [13].

Дори този относително малък REST пример има методологична стойност. Той

показва, че границата между традиционно MVC приложение и API ориентирано приложение не е абсолютно строга. Една добре структурирана система може да комбинира двата подхода и да предоставя различни изгледи върху една и съща домейн логика според нуждите на клиента.

1.2.10. Обобщение на теоретичната рамка

Разгледаните понятия очертават теоретичния фундамент на настоящата разработка. Клиент-сървърният модел определя начина, по който потребителят взаимодейства със системата. Многослойната архитектура организира вътрешните ѝ отговорности. MVC моделът структурира уеб заявките и визуализациите. Обектно-релационното моделиране свързва домейн обектите с базата данни. Валидацията ограничава невалидния вход. Хеширането на пароли и управлението на сесия осигуряват базова сигурност. Thymeleaf и локализацията оформят представящия слой, а REST крайна точка показва алтернативен начин за експониране на данни.

Тази теоретична рамка не е само фон; тя е пряко материализирана в структурата на проекта. Следващата глава преминава от общите понятия към конкретното проектиране на уеб-базираната платформа – изискванията, архитектурата и модела на данните.

2. ПРОЕКТИРАНЕ НА УЕБ-БАЗИРАНАТА ПЛАТФОРМА

Настоящата глава представя проектирането на разработваната веб-базирана платформа. След въвеждането на общия контекст и теоретичната рамка в предходната глава, тук фокусът се пренасочва към конкретните инженерни решения: изискванията към системата, нейната архитектурна организация и домейн моделът, който описва предметната област. Разгледани последователно, тези три аспекта формират проектантската основа на приложението и подготвят прехода към програмната реализация.

2.1. Анализ на изискванията и потребителските сценарии

След въвеждането на общия контекст и теоретичната рамка е необходимо системата да бъде описана от гледна точка на конкретните изисквания, които тя трябва да удовлетворява. Тази стъпка е важна, защото осигурява връзката между предметната област и реалната имплементация. Изискванията не са само списък от желани функции; те са критерий, спрямо който могат да бъдат оценени архитектурата, структурата на кода и степента, в която приложението изпълнява предназначението си.

В случая на онлайн магазин за компютърна и офис техника изискванията се формират както от типичните процеси в електронната търговия, така и от конкретния начин, по който проектът е структуриран в сорс кода. Анализът показва, че системата е ориентирана към сравнително малка, но функционално завършена среда, в която регистрираният потребител може да управлява своята информация, да въвежда фирми и продукти и да преминава през основния цикъл на покупка.

2.1.1. Основни участници в системата

В най-общ вид могат да бъдат разграничени три типа участници, които влияят върху поведението на системата.

Посетител без активна сесия Това е потребител, който достъпва приложението без да е логнат. В този режим системата следва да предоставя начална страница, страница за регистрация и страница за вход. Посетителят няма право да разглежда профилни данни, да добавя компании, да въвежда продукти или да използва количката. От гледна точка на реализирания код тази логика е постигната чрез проверки за наличие на атрибут email в

сесията.

Потвърден регистриран потребител Това е основният активен участник в системата. След потвърждение на регистрацията чрез електронна поща и успешен вход той получава достъп до каталога, профила, списъка с потребители, фирмите, формите за добавяне на фирма и продукт, количката и историята на поръчките. Важно наблюдение е, че в текущия проект липсва отделна роля от тип администратор; много от действията, които в производствена среда биха били ограничени, тук са достъпни за всеки удостоверен потребител. Това е приемливо за учебен сценарий, но трябва да бъде отчетено като особеност на изискванията към настоящата имплементация.

Външни инфраструктурни компоненти Освен човешките потребители системата взаимодейства и с външни компоненти. На първо място това е MySQL базата данни, която съхранява постоянната информация. На второ място това е SMTP сървърът, използван за изпращане на електронно писмо при регистрация. На трето място може да се разглежда и браузърният JavaScript код, който използва REST крайна точка за извличане на максимална цена на продукт. Макар тези компоненти да не са „актьори“ в класически смисъл, те формират съществена част от оперативния контекст на приложението.

2.1.2. Основни потребителски цели

Анализът на предметната област позволява да бъдат формулирани няколко основни потребителски цели:

- създаване на акаунт и потвърждение на електронна поща;
- успешно удостоверяване и работа в персонална сесия;
- въвеждане и поддържане на фирмена информация, асоциирана с потребителя;
- въвеждане на продукти с категория, цена, количество и изображение;
- разглеждане на каталог и избор на подходящ артикул;
- добавяне на избран продукт в количка с желано количество;
- финализиране на поръчка с адрес за доставка;
- преглед на минали поръчки и история на потребителската активност.

Тези цели са практическят еквивалент на функционалните изисквания. Те показват как системата се възприема от потребителя не като набор от контролери и таблици, а като последователност от очаквани резултати.

2.1.3. Функционални изисквания

Функционалните изисквания определят какви операции системата трябва да поддържа. В настоящата разработка те могат да бъдат формулирани по следния начин.

Регистрация и потвърждение на акаунт Системата трябва да позволява създаване на нов потребителски профил чрез форма за регистрация. При успешно подадени данни следва да бъде създаден запис в базата, паролата да бъде съхранена в хеширан вид, а на посочения имейл да бъде изпратена връзка за потвърждение. До потвърждаване на акаунта достъпът до системата не трябва да бъде пълен. В текущата реализация това е постигнато чрез булево поле за потвърден статус и логика в потребителската услуга.

Вход и изход от системата След потвърждаване на акаунта потребителят трябва да може да влезе в системата с имейл и парола. При успешен вход в сесията се съхранява идентификаторът на потребителя под формата на електронна поща, което позволява контролерите да проверяват дали сесията е активна. Системата трябва да поддържа и коректен изход, при който сесията се инвалидира.

Управление на потребителски профил Удостовереният потребител трябва да може да преглежда и редактира основни лични данни. Това включва промяна на име, фамилия, имейл и парола. При смяна на паролата системата следва да изисква въвеждане на старата парола и да проверява коректността ѝ, преди да запази новата стойност в криптиран вид.

Управление на фирми Потребителят трябва да може да въвежда фирми, които впоследствие да се използват при добавяне на продукти. За всяка фирма се поддържат име, ЕИК/идентификатор, ДДС номер, адрес и признак за регистрация. В контекста на текущия проект фирмата е логически свързана с конкретен потребител и се използва като източник на собственост за продуктите.

Управление на продукти Системата трябва да позволява добавяне на нов продукт с наименование, цена, количество, категория, фирма и URL към изображение. Категорията

следва да бъде избрана от предварително дефиниран набор от стойности, което улеснява контрола на данните и последващото филтриране в каталога. След добавяне продуктът трябва да бъде достъпен за визуализация в каталожната част на приложението.

Каталог и преглед на детайли Всеки удостоверен потребител трябва да може да разглежда каталога на наличните продукти. Каталогът трябва да поддържа филтриране по категории, а за всеки продукт да се визуализират име, категория, цена и изображение. При избор на конкретен артикул системата трябва да предоставя детайлен изглед, от който да бъде възможно добавяне към количката.

Количка и финализиране на поръчка След избор на продукт потребителят трябва да може да добави желано количество в количката, ако то е налично. При този процес системата трябва да проверява дали заявеното количество не надвишава текущата наличност. Количката трябва да визуализира избраните артикули, единичната и сумарната цена, както и да поддържа оформяне на поръчка чрез адрес за доставка и калкулация на крайна стойност.

История на поръчките След приключване на покупката системата трябва да съхрани исторически запис за направената поръчка, включително дата, адрес и обща стойност. Потребителят трябва да може да разглежда тези записи в отделен екран и при необходимост да изчиства натрупаната история.

Локализация и допълнителен REST интерфейс Системата трябва да поддържа базова смяна на езика между английски и български за част от интерфейса чрез локализирани съобщения. Освен това трябва да предлага поне една примерна REST крайна точка, която връща агрегирана информация в машинно четим формат. В текущия проект това е информацията за максималната цена на продукт.

Таблица 2. Обобщение на основните функционални изисквания

Идентификатор	Изискване	Реализация в проекта
FR-1	Регистрация на потребител	Форма за регистрация, запис в базата, хеширане на парола
FR-2	Потвърждение на акаунт	Имейл с линк и поле <code>isConfirmed</code>
FR-3	Вход и изход	Проверка на парола и управление на <code>HttpSession</code>
FR-4	Редакция на профил	Форма за профил и проверка на стара парола
FR-5	Добавяне на фирма	Отделен контролер и <code>binding</code> модел за фирмени данни
FR-6	Добавяне на продукт	Категории, фирма, количество, цена и URL изображение
FR-7	Разглеждане на каталог	Категорийни маршрути и Thymeleaf визуализация
FR-8	Добавяне в количка	Детайлен изглед, количество и проверка за наличност
FR-9	Финализиране на поръчка	Адрес, обща цена и създаване на история
FR-10	История на поръчките	Екран с исторически записи и операция за изчистване
FR-11	Локализация	<code>messages.properties</code> , <code>messages_bg.properties</code> , <code>interceptor</code>
FR-12	REST услуга	Крайна точка за максимална цена на продукт

Таблица 2 показва, че функционалностите на системата следват логична

последователност от регистрация и вход към управление на данни, покупка и проследяване на резултата от тази покупка. Това е именно цикълът, който определя проекта като завършено макар и компактно електронно търговско приложение.

2.1.4. Нефункционални изисквания

Наред с конкретните функции, системата следва да отговаря и на редица нефункционални изисквания. Те не описват какво прави приложението, а как следва да го прави.

Поддържаемост Кодът трябва да бъде организиран така, че логиката на системата да бъде лесна за проследяване. Това предполага разделяне по пакети, отделяне на контролери, услуги и repository компоненти, както и използване на отделни модели за вход, вътрешен трансфер и визуализация. Поддържаемостта е особено важна в дипломна разработка, тъй като документът трябва да може ясно да обясни структурата на проекта.

Четим и последователен потребителски интерфейс Системата трябва да предлага интерфейс, който е достатъчно интуитивен за базови потребителски операции. Макар дизайнът да не е водещ изследователски фокус, от приложението се очаква да поддържа последователна навигация, разбираеми форми и ясни маршрути към основните действия.

Коректност на входните данни Всички основни форми следва да бъдат защитени чрез валидация, така че очевидно невалидни стойности да не достигат до persistence слоя. В разглеждания проект това се реализира чрез анотации и обработка на резултата от bind операцията.

Устойчиво съхранение на информацията Приложението трябва да използва релационна база данни, за да гарантира, че потребители, фирми, продукти, количка и история се съхраняват устойчиво и са достъпни между отделни изпълнения на системата. Използването на MySQL и Hibernate осигурява именно този тип устойчивост.

Елементарна сигурност Необходимо е паролите да бъдат съхранявани хеширано, а достъпът до определени страници да бъде ограничен до удостоверени потребители. Макар това да не реализира пълноценна корпоративна схема за сигурност, то е минимално необходимо условие, за да бъде системата функционално и методологично коректна.

Разширяемост Архитектурата следва да позволява добавяне на нови категории, нови екрани и нови услуги без фундаментално пренаписване на вече изградената основа. Именно поради това изборът на многослоен модел и Spring компоненти има не само настоящ, но и стратегически смисъл.

2.1.5. Ключови потребителски сценарии

За по-голяма яснота функционалните изисквания могат да бъдат представени и като сценарии. Това е полезно, защото показва не просто кои функции съществуват, а как те се комбинират в завършен работен поток.

Сценарий 1: Регистрация и първи вход Посетителят отваря началната страница и избира регистрация. Той попълва име, фамилия, имейл и парола. След успешно изпращане системата създава нов потребител, хешира паролата и изпраща линк за потвърждение. След активиране на акаунта потребителят се връща към формата за вход, въвежда данните си и системата създава активна сесия.

Сценарий 2: Добавяне на фирма и продукт След успешен вход потребителят отваря екрана за добавяне на фирма и въвежда необходимите атрибути. След това преминава към формата за нов продукт, избира една от наличните категории, посочва фирмата, задава цена, количество и URL на изображение. След запис продуктът става част от каталога.

Сценарий 3: Разглеждане на каталог и покупка Потребителят разглежда каталога, филтрира по категория и избира конкретен продукт. Отваря детайлния екран, въвежда желано количество и го добавя в количката. Ако количеството е налично, системата актуализира наличността и създава или обновява запис в количката. След това потребителят отваря количката, въвежда адрес за доставка и потвърждава поръчката. Системата създава исторически запис и изчиства активната количка.

Сценарий 4: Преглед на история и профил Във всеки момент удостовереният потребител може да отвори профила си, да редактира лични данни и да провери асоциираните компании. Той може да отвори и историята на поръчките, където да види натрупаните записи и при необходимост да ги изчисти.

2.1.6. Ограничения, произтичащи от текущата постановка

Анализът на изискванията разкрива и някои характерни ограничения. Липсата на отделни роли означава, че системата не разграничава административни от потребителски действия. Локализацията е частична и не обхваща изцяло всички текстове и категории. Някои операции, които в production среда биха били защитени чрез по-строга политика, тук са реализирани с по-прост модел. Същевременно тези ограничения не обезсмислят проекта; напротив, те очертават ясна граница между учебна система и пълноценна комерсиална платформа.

2.1.7. Изводи

Изискванията към системата показват, че проектът е замислен като компактно, но завършено уеб приложение, покриващо най-важните процеси на един онлайн магазин. Формулираните функционални и нефункционални критерии създават здрава основа за анализа на архитектурата и кода в следващите раздели. Особено важно е, че изискванията са достатъчно конкретни, за да позволят последваща верификация, и достатъчно реалистични, за да направят системата представителна за практическо приложение в областта на електронната търговия.

2.2. Архитектура на системата

След формулирането на изискванията естествената следваща стъпка е анализ на архитектурната организация на приложението. Архитектурата е гръбнакът на всяка нетривиална система, защото определя как отделните отговорности са разпределени, как се движат данните между модулите и до каква степен проектът може да се поддържа и развива във времето. В настоящата разработка архитектурният избор е насочен към ясно разделение между уеб слой, бизнес логика, достъп до данни, конфигурация и визуализация. Този раздел описва именно това разпределение и го свързва с реалната структура на сорс кода.

2.2.1. Обща архитектурна постановка

Системата е реализирана като монолитно уеб приложение върху Spring Boot. Терминът „монолитно“ тук не следва да се възприема като негативен. В контекста на настоящия проект той означава, че всички основни функционални компоненти – уеб

Фигура 2 показва, че приложението следва централен уеб поток, но може да предоставя резултати както чрез HTML визуализация, така и чрез JSON представяне. Това е важен белег за архитектурна гъвкавост, въпреки че системата е доминиращо сървърно рендирана.

2.2.2. Пакетна организация на сорс кода

Основният Java код е разположен в директорията `src/main/java/com/example/diplomenproekt`. Вътре в нея проектът е разделен на няколко логически под-пакета. Тази организация не е просто въпрос на стил; тя изразява пряко архитектурното разделение на отговорностите.

Таблица 3. Пакети в Java слоя и тяхното предназначение

Пакет	Основно предназначение
<code>config</code>	Съдържа конфигурационни класове за bean дефиниции и локализация
<code>models.entities</code>	Описва JPA entity обектите на домейна
<code>models.bindingModel</code>	Съдържа модели за приемане и валидация на входни форми
<code>models.serviceModel</code>	Описва междинни модели за пренос на данни към service слоя
<code>models.viewModel</code>	Съдържа модели, предназначени за визуализация в шаблоните или API
<code>repositories</code>	Дефинира интерфейсите за достъп до базата данни чрез Spring Data JPA
<code>services</code>	Съдържа service интерфейси, описващи бизнес възможностите
<code>services.impl</code>	Реализира бизнес логиката и координацията между компонентите
<code>web</code>	Съдържа MVC и REST контролери за входящите HTTP заявки

От таблица 3 се вижда, че дори при сравнително малък проект е постигната

смыслена пакетна сегрегация. Това е едно от силните качества на разработката, защото улеснява навигацията, поддръжката и аналитичното описание на системата.

2.2.3. Входна точка и стартиране на приложението

Входната точка на системата е класът `DiplomenProektApplication`, маркиран с анотация `@SpringBootApplication`. Тази анотация обединява механизми за component scanning, auto-configuration и bootstrapping на приложението [4]. На практика това означава, че при стартиране Spring открива контролерите, услугите, repository интерфейсите и конфигурационните класове, създава необходимите bean инстанции и подготвя средата за обслужване на заявки.

От архитектурна гледна точка е важно, че входният клас не съдържа бизнес логика. Той изпълнява единствено ролята на стартова точка. Това е добра практика, защото предотвратява смесването на инфраструктурна и домейн логика още на най-високо ниво на приложението.

2.2.4. Конфигурационен слой

В пакета `config` са разположени два съществени класа: `ApplicationBeanConfiguration` и `I18nConfig`. Първият дефинира bean компоненти за `ModelMapper` и `PasswordEncoder`. Тук архитектурната идея е ясна: компоненти с инфраструктурна роля, използвани на множество места в системата, трябва да се управляват централизирано от Spring контейнера, а не да се създават ръчно във всеки клас.

`I18nConfig` от своя страна настройва `LocaleResolver` и `LocaleChangeInterceptor`, с което създава базов механизъм за локализация на интерфейса. Това показва, че архитектурата не е ограничена единствено до CRUD логика, а обхваща и допълнителни крос-срезови характеристики на приложението.

Централизираната конфигурация има и друго предимство: когато даден инфраструктурен избор трябва да бъде променен, това става на едно определено място. Така например подмяна на password encoder или промяна на локализационния механизъм не изисква търсене и редакция в множество несвързани класове.

2.2.5. Уеб слой: контролери и маршрути

Уеб слойът е реализиран в пакета `web`. В него се намират контролерите `HomeController`, `UserController`, `CompanyController`, `ProductController` и `MaxPriceRestController`.

oller. Разделянето им по функционални области е логично и съответства на основните домейн сценарии на системата.

HomeController обслужва началната точка на приложението и реализира проста логика за пренасочване към каталога при активна сесия. UserController е най-богатият по отношение на потоци компонент, защото управлява регистрацията, входа, изхода, профила, списъка с потребители и потвърждението на електронна поща. CompanyController обслужва формите за въвеждане на фирма. ProductController управлява продуктовия каталог, добавянето на продукт, детайлния изглед, количката, checkout процеса и историята на поръчките. MaxPriceRestController предоставя отделен JSON интерфейс.

От архитектурна гледна точка силна страна на проекта е, че контролерите до голяма степен запазват ролята си на координатори, а не на място за тежка бизнес логика. Те валидират достъпа до маршрута, проверяват резултата от binding операцията, извикват съответна услуга и определят към коя view да премине изпълнението. Това следва добрата практика контролерът да бъде „тънък“ и ориентиран към уеб потока, а не към самото бизнес правило.

2.2.6. Service слой: сърце на бизнес логиката

Service слойът е организиран чрез интерфейси и имплементации. Това дава две важни предимства. Първо, архитектурно се разграничават договорът и конкретната реализация. Второ, системата става по-подходяща за бъдещо тестване и подмяна на конкретни имплементации.

В настоящия проект могат да бъдат разграничени следните основни service области:

- потребителски операции – регистрация, вход, потвърждение, редакция на профил, изчисляване на обща цена в количката;
- управление на фирми – добавяне и извличане на фирми за конкретен потребител;
- управление на продукти – добавяне, категоризация, търсене по идентификатор, проверка за наличност, добавяне в количка;
- количка – съхранение и извличане на записи за избрани продукти;
- история – финализиране на поръчка, изчистване и извличане на минали покупки.

Service слойът е ключов за архитектурата, защото именно там се вземат решения за последователността на операциите. Например при checkout процеса е необходимо не просто да се създаде исторически запис, а да се координира обновяването на количката, историята и продуктите. Този тип действия са типичен пример за логика, която не принадлежи нито на контролера, нито на repository интерфейса, а на самия бизнес слой.

2.2.7. Persistence слой и repository компоненти

Persistence слойът е реализиран чрез Spring Data JPA repository интерфейси: UserRepository, CompanyRepository, ProductRepository, ShoppingCartRepository и HistoryRepository. Тяхната основна функция е да предоставят стандартизиран и сравнително декларативен достъп до данните.

Наличието на методи като findByEmail, getAllByUser_Id, findAllByUser_Id и deleteAllByUser_Id показва типичния за Spring Data подход: част от необходимата query логика се изразява директно чрез имената на методите, без ръчно писане на SQL. Това спестява boilerplate код и прави част от persistence слоя по-компактен.

От архитектурна перспектива repository слойът е правилно ограничен до операции по извличане, запис и изтриване. Той не съдържа самостоятелна бизнес логика, а служи като механизъм, чрез който service слойът реализира домейн правилата. Именно това разделение е един от важните признаци за архитектурна последователност.

2.2.8. Модели за трансфер на данни между слоевете

Интересна архитектурна особеност на проекта е използването на няколко типа модели:

- entity модели за persistence слой;
- binding модели за входни форми;
- service модели за междинен трансфер;
- view модели за визуализация и API представяне.

Този подход съответства на добрите практики за ограничаване на директната зависимост между базата данни и потребителския интерфейс. Например не е необходимо HTML шаблоните да получават винаги самите entity обекти; вместо това те могат да работят с view модели, съдържащи само необходимите за визуализация полета.

Аналогично, при приемане на данни от форма е по-удачно да се използва binding модел с валидационни анотации, отколкото директно persistence обектът.

Използването на `ModelMapper` подсказва стремеж към намаляване на ръчните преобразувания между тези представяния. Това е полезно от инженерна гледна точка, но не отменя необходимостта от внимателно моделиране на самите класове и полета.

2.2.9. Представящ слой: шаблони и статични ресурси

Визуалният слой на приложението е разположен в `src/main/resources/templates` и `src/main/resources/static`. Шаблоните отразяват основните потребителски екрани: начална страница, регистрация, вход, профил, добавяне на фирма, добавяне на продукт, каталог, детайли, количка, история и страници за грешки. Статичните ресурси включват CSS и JavaScript файлове, както и изображения.

Архитектурното значение на този слой се състои в това, че той остава относително изолиран от persistence логиката. Шаблоните работят с вече подготвени атрибути от модела и не съдържат сложни бизнес решения. Това е важен белег за дисциплинирано разделение на отговорностите.

2.2.10. Архитектурен поток на ключови операции

За да бъде по-ясно как слоевете си взаимодействат, е полезно да се разгледа общ модел на няколко основни операции.

Регистрация При регистрация браузърът изпраща POST заявка към потребителския контролер. Контролерът приема binding модел и го предава към потребителската услуга. Услугата проверява дали вече съществува потребител с този имейл, създава entity обект, хешира паролата, записва новия потребител чрез repository слоя и изпраща потвърждаващ имейл чрез инфраструктурен компонент. След това контролерът връща подходящо пренасочване.

Добавяне на продукт Контролерът за продукти валидира входната форма и предава данните към `ProductsService`. Услугата намира избраната фирма, създава нов продукт, преобразува категорията от низ към `CategoryEnum` и записва обекта в базата. По този начин контролерът остава концентриран върху маршрута и валидацията, а service слоят поема домейн решението.

Checkout Checkout сценарият е най-показателен за архитектурната координация. Контролерът приема адрес и обща цена, валидира входа и извиква `HistoryService`. Тази услуга създава запис в историята, асоциира го с потребителя, нулира или изчиства данните за текущата количка и премахва продукти с нулева наличност. Тук ясно се вижда защо `service` слойът е необходим: операцията изисква координация между няколко `persistence` компонента и не може да бъде сведена до една `repository` заявка.

2.2.11. Силни страни на архитектурното решение

От направения анализ могат да бъдат изведени няколко силни страни на архитектурата:

- ясна пакетна организация, която отразява логическите роли на класовете;
- разграничение между контролери, услуги и `repository` интерфейси;
- използване на отделни модели за различни етапи на обмен на данни;
- централизация на инфраструктурни компоненти чрез конфигурационни `bean` дефиниции;
- наличие както на `HTML`, така и на примерен `REST` интерфейс.

Тези характеристики правят системата подходяща не само за изпълнение на текущата функционалност, но и за последващ анализ и разширение.

2.2.12. Архитектурни компромиси и ограничения

Наред със силните страни е необходимо да бъдат отчетени и определени компромиси. Част от проверките за достъп са реализирани директно в контролерите чрез условие върху сесията, вместо чрез централизирана `security` архитектура. Това прави системата по-лесна за разбиране, но по-слаба откъм формална сигурност. Някои операции в `service` слоя показват силна зависимост от конкретната структура на данните, което е очаквано за малък проект, но при по-голямо приложение би следвало да бъде допълнително абстрахирано.

Допълнително, макар архитектурата да е многослойна, тя все още е тясно обвързана с конкретната реализация на сесиен достъп и локална конфигурация. Това е разумно за учебен сценарий, но следва да бъде адресирано при бъдещо развитие към по-зряла среда.

2.2.13. Архитектурно обобщение

Архитектурата на разглежданата система представлява добре подредена монолитна Spring Boot реализация с ясно разграничени слоеве и пакетна структура. Използваният модел е напълно адекватен за домейн като онлайн магазин с ограничен, но реалистичен обхват. Той позволява логическа проследимост на данните и операциите, създава добра основа за поддръжка и надграждане и подчертава учебно-инженерната стойност на разработката.

2.3. Домейн модел и модел на данните

Домейн моделът е централната абстракция, чрез която софтуерната система представя предметната област. В разглеждания проект той описва основните участници и артефакти в процеса на електронна търговия: потребител, фирма, продукт, количка и история на поръчки. Макар приложението да не реализира сложна бизнес семантика от enterprise клас, домейн моделът е достатъчно богат, за да наложи ясно картографиране към релационна база данни и разделяне на различни типове модели за обработка и визуализация на данни.

Настоящият раздел разглежда едновременно логическото съдържание на домейн обектите и начина, по който те са реализирани чрез JPA entity класове, binding модели, service модели и view модели. По този начин се проследява целият път на информацията: от въвеждането ѝ във форма, през вътрешното ѝ представяне в системата, до записването ѝ в базата и повторната ѝ визуализация.

2.3.1. Основни домейн понятия

В предметната област на електронния магазин могат да бъдат идентифицирани няколко базови понятия.

- **Потребител** – физическо лице, което се регистрира, влиза в системата, поддържа профил, добавя фирми, въвежда продукти, използва количка и има история на поръчки.
- **Фирма** – обект, който представя търговска или доставна организация, асоциирана с конкретен потребител и използвана при добавяне на продукти.
- **Продукт** – артикул от каталога, описан чрез име, цена, наличност, изображение и категория.

- **Запис в количка** – конкретно желание за покупка на определен продукт в дадено количество.
- **Исторически запис** – обобщена информация за приключена поръчка, включваща адрес, дата и цена.

Тези понятия образуват ядрото на домейна и определят по-голямата част от класовете в `models.entities`. От гледна точка на *domain-driven* мисленето [14], те могат да бъдат разглеждани като основните бизнес обекти, около които се организират и останалите механизми в приложението.

2.3.2. Entity моделът в JPA слоя

Persistence слойът използва JPA entity класове, които наследяват `BaseEntity`. Макар в текущата работа да не е необходимо да се навлиза в детайли на базовия клас, неговото наличие подсказва стандартизирано централизиране на идентификатора за всички трайни обекти. Това е типична практика и намалява дублирането на общи persistence характеристики.

Потребител Класът `User` е основният aggregate обект в системата. Той съдържа име, фамилия, парола, признак дали акаунтът е потвърден и електронна поща. Освен това поддържа връзки към фирмите на потребителя, към записите в количката и към историята на поръчките. Така именно потребителят е логическият център на множество операции в приложението.

Особен интерес представлява полето за електронна поща. То е маркирано като уникално и допълнително има регулярен израз на ниво entity, което подсказва стремеж към защита на едно от най-важните идентифициращи полета. Паролата не се валидира на това ниво, а се обработва и хешира чрез service логика. Булевото поле за потвърждение е част от модела на регистрация и контролира допустимостта на входа.

Фирма Класът `Company` описва фирмена информация и включва атрибути като име, `vat`, адрес и признак за регистрация. В системата фирмата принадлежи на конкретен потребител чрез връзка `@ManyToOne`. Освен това една фирма има множество продукти чрез `@OneToMany(mappedBy="company")`. От логическа гледна точка това е разумен модел: фирмата изпълнява ролята на притежател или източник на предлаганите продукти.

Следва да се отбележи, че името на колекцията в класа `Company` е `companies`, въпреки че тя реално съдържа продукти. Това е незначително от функционална гледна точка, но е показателно за важността на консистентното именуване в домейн модела. Подобни дребни несъответствия не чупят системата, но намаляват концептуалната яснота и следва да бъдат разглеждани като потенциален обект на бъдещо рефакториране.

Продукт Класът `Product` е основният каталожен обект в приложението. Той съдържа име, цена, количество, URL към изображение, категория и връзка към фирма. Категорията е реализирана като изброим тип `CategoryEnum`, което е добър архитектурен избор. По този начин наборът от допустими категории е изрично описан в кода и не зависи от свободен текст или произволни стойности, въведени от потребителя.

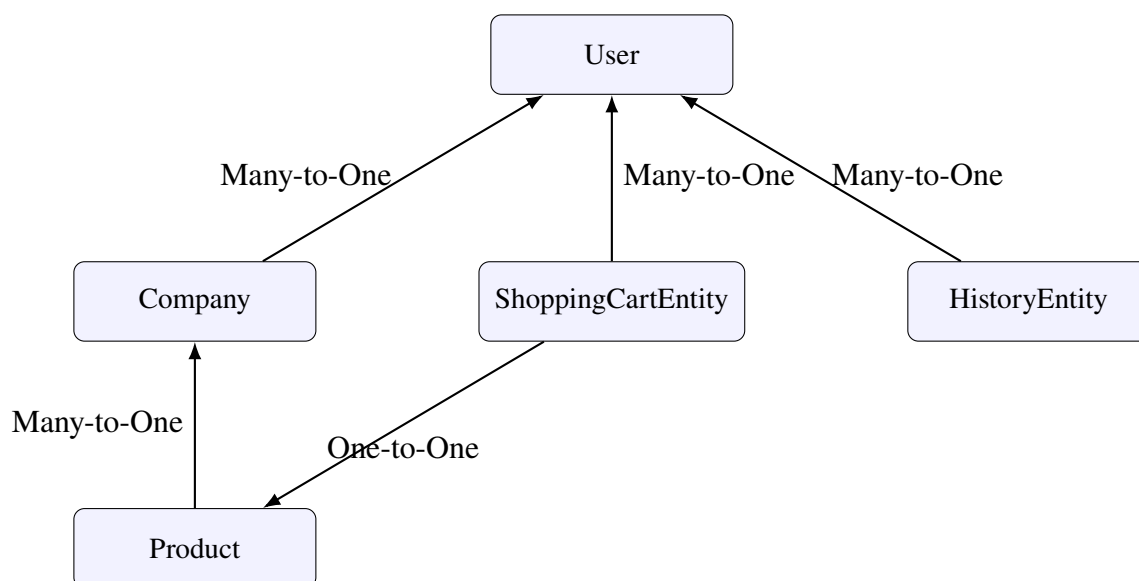
Валидирането на цена и количество е реализирано чрез `@Min(0)`, което формализира базовото бизнес правило, че продуктите не могат да имат отрицателни стойности за тези полета. От бизнес гледна точка продуктът е едновременно обект на визуализация и на транзакционно взаимодействие, защото участва в количката и се променя при checkout процеса.

Запис в количка Класът `ShoppingCartEntity` моделира преходно търговско състояние: конкретен потребител е избрал конкретен продукт в конкретно количество. Той съдържа само три основни атрибута – желано количество, продукт и потребител. Макар моделът да е опростен, той е достатъчен за целите на проекта и илюстрира идеята, че количката не е просто списък от продукти, а списък от връзки „потребител–продукт–брой“.

Интересен детайл е, че продуктът в количката е реализиран с `@OneToOne`, а потребителят с `@ManyToOne`. Това позволява множество записи в количката да принадлежат на един потребител, но ограничава логическата връзка към конкретен продукт. В production сценарий често би се използвала и различна схема за моделиране на поръчките позиции, но за настоящия проект този модел е достатъчен, за да представи основната идея.

История на поръчките Класът `HistoryEntity` представя обобщен запис за реализирана поръчка. В него се съхраняват адрес, дата, цена и връзка към потребител. Това е умишлено опростен модел. Вместо поръчката да се разглежда като самостоятелен aggregate с отделни позиции, статуси и платежни детайли, тук тя е сведена до

исторически запис за случило се checkout събитие. Подобен подход е напълно оправдан в учебно-приложен контекст, тъй като позволява системата да покаже завършен цикъл на покупка без да навлиза в сложността на пълномащабен order management модел.



Фигура 3. Обобщена схема на връзките между основните entity обекти

Фигура 3 представя опростено отношенията между основните трайни обекти. От нея ясно се вижда, че потребителят е централният домейн елемент, към който се асоциират фирмите, количката и историята, докато продуктите са свързани с фирмите и се използват в количката.

2.3.3. Атрибутен анализ на основните entity класове

За по-систематичен преглед е полезно ключовите класове да бъдат сравнени по техните атрибути и роли в системата.

Таблица 4. Атрибути и роля на основните entity класове

Клас	Основни атрибути	Роля в системата
User	firstName, lastName, password, email, isConfirmed	Представя идентичността на потребителя и служи като входна точка към фирми, количка и история
Company	name, mol, vat, address, isRegistered	Представя фирмен субект, който се асоциира с потребителя и към който принадлежат продукти

Клас	Основни атрибути	Роля в системата
Product	name, price, quantity, url, category	Представя каталоген артикул, който може да бъде визуализиран, филтриран и закупуван
ShoppingCart Entity	wishedQuantity ForOrder, връзка към продукт и потребител	Представя временно потребителско намерение за покупка
HistoryEntity	address, date, price, връзка към потребител	Представя обобщение на финализирана поръчка

Таблица 4 потвърждава, че домейн моделът е достатъчно компактен, но и достатъчно изразителен. В него няма излишни слоеве на абстракция, а всеки entity клас има ясно разпознаваема роля.

2.3.4. Изброим тип за категориите

Категоризационният модел е реализиран чрез CategoryEnum. В него са дефинирани стойности като VIDEOCARDS, PROCESSORS, HEADPHONES, PRINTERS, OFFICE_CHAIRS, SCANNERS, ROUTERS, MICE и KEYBOARDS. Този избор е особено подходящ по три причини.

Първо, елиминира се рискът от невалидни категории, въведени чрез свободен текст. Второ, контролерът за каталог може да използва ясни branch условия по категории. Трето, категориите се превръщат в част от самия домейн модел, а не остават външен конфигурационен списък без типова защита.

Цената на това решение е, че добавянето на нова категория изисква промяна в кода и повторна компилация. За текущия проект това не е проблем, тъй като категориите са предварително известни и тяхната стабилност е по-важна от динамичната конфигурируемост.

2.3.5. Binding модели и входна валидация

Съществена част от домейн обработката в системата не се извършва директно чрез entity класовете, а чрез binding модели. Това са класове, предназначени за приемане на данни от HTML форми. Сред тях са UserRegisterBindingModel, UserLoginBindingMo

del, ProfileUpdateBindingModel, AddCompanyBindingModel, ProductAddBindingModel, AddToCartBindingModel и CheckoutBindingModel.

От архитектурна гледна точка тези класове изпълняват две роли. Първо, дефинират структурата на потребителския вход. Второ, концентрират на едно място част от валидационната логика чрез анотации като @Size, @NotBlank, @NotNull и @Positive [10]. Това прави формите по-прогнозируеми и намалява вероятността до service слоя да достигнат очевидно невалидни данни.

Таблица 5. Примери за валидационни ограничения в binding моделите

Модел	Ограничения	Смисъл на ограничението
ProfileUpdateBindingModel	@Size(min=3,max=20) за име, фамилия, имейл и пароли	Ограничаване на очевидно невалидни или прекомерно кратки стойности
AddCompanyBindingModel	@NotBlank, @Size, @Positive, @NotNull	Валидиране на фирмено име, адрес и числови идентификатори
ProductAddBindingModel	@NotBlank, @Positive, @NotNull	Гарантира коректни данни за продукт преди запис
CheckoutBindingModel	@Size(min=5,max=20) за адрес	Базова проверка на адреса при финализиране на поръчка

Макар този модел да е полезен, следва да се отбележи, че не всички binding класове в проекта имат еднакво богата валидация. Например регистрационният и login моделът са по-леки и не носят симетричен набор от ограничения. Това е напълно допустимо в учебна разработка, но би могло да бъде разширено при бъдещо развитие.

2.3.6. Service и view модели

Освен entity и binding класове проектът използва и service модели, и view модели. Service моделите служат като вътрешно представяне между persistence и service слоя, а view моделите са оптимизирани за визуализация в шаблоните или за връщане през API. Примери за такива класове са UserServiceModel, UserLoginServiceModel, CompanyServiceModel, CompanyViewModel, ProductViewModel, UserViewModel и MaxPriceViewModel.

Предимството на този подход е, че отделните слоеве могат да работят с различни представяния на едни и същи логически данни, без да бъдат принудени да използват директно entity обектите навсякъде. Това подобрява капсулацията. Например шаблонът за каталог не е длъжен да получава пълния entity обект на продукта, а само тази част от информацията, която е необходима за визуализация.

2.3.7. Преобразувания между моделите

Преобразуването между различните представяния е подпомогнато от ModelMapper. Тази библиотека намалява ръчния код при преобразуване на entity към service модел или на service модел към view модел. В контекста на проекта това улеснява реализацията на списъци с потребители, фирми и продукти. Въпреки това част от домейн логиката остава ръчно реализирана, например когато е необходимо да се създаде изцяло нов entity обект с попълване на конкретни полета и зависимости.

Използването на марпег инструмент е особено полезно при средни по размер проекти. То намалява повторяемия код, но същевременно не скрива домейн решенията, когато те са специфични. Именно такъв е случаят тук: стандартните трансформации се автоматизират, а смислово важните операции се извършват експлицитно в service слой.

2.3.8. Съответствие между домейн модел и потребителски сценарии

Една от силните страни на разглеждания проект е, че домейн моделът сравнително пряко следва основните потребителски сценарии. Потребителят създава фирма; фирмата притежава продукти; продуктът се добавя в количката; количката участва във финализиране на поръчка; поръчката се архивира в историята. Тази линия е достатъчно ясна, за да бъде следвана както от кода, така и от документацията.

Разбира се, моделът не е напълно независим от ограниченията на имплементацията. Историята не съдържа отделни позиции по поръчка, а количката е опростена спрямо типична production система. Въпреки това домейнът е достатъчно консистентен, за да изпълни предназначението си като основа на учебно-приложен онлайн магазин.

2.3.9. Критични наблюдения върху модела

Анализът на модела позволява да бъдат формулирани и няколко критични наблюдения:

- налице са дребни несъответствия в именуването на някои свойства, което затруднява

концептуалната чистота;

- поръчката е представена в силно опростен вид чрез исторически запис, без отделни order item обекти;
- някои валидационни правила са концентрирани само върху част от binding моделите;
- моделът е силно ориентиран към един потребителски тип и не включва роли или права.

Тези наблюдения не обезсилват решението, а по-скоро очертават границите на неговата текуща зрялост. Именно поради това те са ценни за документацията: показват, че системата е анализирана критично, а не само описателно.

2.3.10. Обобщение на проектирането

Проектирането на уеб-базираната платформа съчетава ясно дефинирани изисквания, добре подредена многослойна архитектура и компактен, но изразителен домейн модел. Тези три аспекта формират основата, върху която стъпва програмната реализация, разгледана в следващата глава.

3. ПРОГРАМНА РЕАЛИЗАЦИЯ

След представянето на изискванията, архитектурата и домейн модела следва анализ на конкретната програмна реализация. Тази глава разглежда последователно три аспекта на изграденото приложение: реализацията на back-end логиката чрез контролери, услуги и repository компоненти; представящия слой и потребителското взаимодействие чрез Thymeleaf шаблони; и операционната среда – структурата на проекта, конфигурацията и build процеса. Целта е архитектурните идеи от предходната глава да бъдат проследени в тяхната програмна материализация.

3.1. Реализация на back-end логиката

След описването на архитектурата и домейн модела е необходимо да се разгледа конкретната имплементация на back-end логиката. В този раздел фокусът е върху действителното поведение на системата, реализирано чрез контролери, услуги, repository компоненти и конфигурация. Анализът следва структурата на проекта и показва как отделните модули взаимодействат, за да реализират регистрация, управление на фирми и продукти, работа с количка, checkout и история на поръчките.

За разлика от предходните глави, които се концентрират върху общите принципи, настоящият раздел стъпва директно върху наличния сорс код. Именно това прави изложението особено ценно: тук архитектурните идеи се виждат в тяхната програмна материализация.

3.1.1. Конфигурация на основните инфраструктурни компоненти

Back-end поведението на приложението зависи от няколко инфраструктурни компонента, дефинирани централизирано. Най-важните сред тях са `ModelMapper` и `PasswordEncoder`. Те се регистрират като Spring bean обекти и впоследствие се инжектират в service компонентите.

```
1 @Configuration
2 public class ApplicationBeanConfiguration {
3
4     @Bean
5     public ModelMapper modelMapper() {
6         return new ModelMapper();
7     }
8 }
```

```

9      @Bean
10     public PasswordEncoder passwordEncoder() {
11         return new Pbkdf2PasswordEncoder();
12     }
13 }

```

Listing 1: Конфигурация на mapper и password encoder

Листинг 1 показва добър пример за централизирано управление на зависимости. ModelMapper подпомага преобразуването между различни модели, а Pbkdf2PasswordEncoder осигурява по-сигурно съхранение на паролите. Макар тази конфигурация да е кратка, тя има ключово значение за правилното функциониране на регистрацията и профилния модул.

3.1.2. Потребителски модул

Потребителският модул е най-обемен от гледна точка на уеб потоци. Той обхваща регистрация, потвърждение на акаунт, вход, изход, извличане на потребителска информация, редакция на профил и изчисляване на стойността на продуктите в количката.

Регистрация на потребител Регистрационният процес започва в UserController, където POST маршрутът за регистрация приема UserRegisterBindingModel. Ако входните данни са коректни от гледна точка на binding и няма вече съществуващ потребител със същия имейл, контролерът делегира работата към UserServiceImpl. Там се създава нов entity обект от тип User, инициализират се колекциите за компании, количка и история, задават се основните атрибути и паролата се хешира преди запис.

След запис на потребителя системата изпраща електронно писмо с линк за потвърждение. Това е съществена част от логиката, защото не позволява автоматичен вход преди потвърждаване на акаунта. От чисто инженерна гледна точка модулет демонстрира добра последователност: контролерът управлява потока, услугата реализира бизнес логиката, а repository слойът извършва persistence операцията.

```

1  @Override
2  public Optional<UserRegisterBindingModel> register(
3      UserRegisterBindingModel input) {
4      if (userRepository.findByEmail(input.getEmail()).isPresent()) {
5          return Optional.empty();
6      }
7
8      User user = new User();

```

```

9      user.setCompanies(new ArrayList<>());
10     user.setProductsCart(new ArrayList<>());
11     user.setHistoryProducts(new ArrayList<>());
12     user.setFirstName(input.getFirstName());
13     user.setLastName(input.getLastName());
14     user.setEmail(input.getEmail());
15     user.setConfirmed(false);
16     user.setPassword(passwordEncoder.encode(input.getPassword()));
17     user = userRepository.saveAndFlush(user);
18
19     SimpleMailMessage message = new SimpleMailMessage();
20     message.setTo(input.getEmail());
21     message.setSubject("Email Confirmation");
22     message.setText(
23         "http://localhost:8080/users/"
24         + user.getId()
25         + "/confirm-email");
26     emailSender.send(message);
27
28     return Optional.of(input);
29 }

```

Listing 2: Опростен вид на регистрационната логика

Листинг 2 илюстрира същината на регистрационния процес. В него ясно се вижда връзката между persistence слоя, сигурността на паролите и e-mail интеграцията. Същевременно могат да се забележат и някои ограничения: URL адресът за потвърждение е твърдо зададен към localhost:8080, а подателят и конфигурацията на пощата зависят от локални настройки.

Вход в системата Входът също е реализиран в UserController. Контролерът подава входния модел към потребителската услуга, която намира потребителя по имейл, проверява дали акаунтът е потвърден и сравнява въведената парола с хеша в базата. При успех имейлът се записва в HttpSession. Този подход е относително прост, но напълно достатъчен за демонстрационен проект.

Важно е да се отбележи, че логиката не използва Spring Security filter chain или security context. Вместо това достъпът до защитени ресурси се контролира ръчно чрез проверки за наличен атрибут в сесията. От методологична гледна точка това прави кода лесен за следване, но въвежда по-голям риск от дублиране на проверки и от пропуски при добавяне на нови маршрути.

Потвърждение на акаунт и управление на профил Потвърждението на акаунта е реализирано чрез маршрут `/users/{userId}/confirm-email`. При извикване на този маршрут потребителската услуга намира съответния потребител и маркира профила като потвърден. След това приложението връща екран за вход.

Профилният модул позволява извличане и обновяване на потребителските данни. Логиката включва две важни проверки: дали текущата сесия е активна и дали старата парола съвпада със съхранения хеш. По този начин системата реализира базов контрол върху промяната на чувствителна информация.

3.1.3. Модул за фирми

Управлението на фирмите е реализирано чрез `CompanyController` и `CompanyServiceImpl`. От функционална гледна точка този модул играе ролята на подготвителен етап за продуктовия каталог, защото всяка фирма може да бъде използвана при въвеждане на продукти.

Контролерът за фирми е относително компактен. Той осигурява GET маршрут за визуализация на формата и POST маршрут за запис на фирма. При POST обработката се използва `AddCompanyBindingModel`, който валидира полета като име, адрес, ДДС номер и регистрационен статус. Service имплементацията намира текущия потребител по имейл от сесията, създава entity обект `Company`, интерпретира текстовото поле за регистрационен признак и инициализира асоциираната колекция от продукти.

Тук се вижда интересна особеност на дизайна: фирмата не е абстрактна част от глобален каталог, а принадлежи към конкретен потребител. Това подсилва идеята, че приложението е по-близо до персонален търговски workspace, отколкото до многостранна публична маркетплейс система.

3.1.4. Модул за продукти и каталог

Продуктовият модул е ключова част от приложението, защото именно чрез него системата придобива своята каталожна функционалност. Този модул е разделен между `ProductController`, `ProductsService` и `ProductServiceImpl`. Освен това разчита на `CompanyRepository`, `ProductRepository`, `UserRepository` и `ShoppingCartService`.

Добавяне на продукт Добавянето на продукт започва с форма, която очаква име, цена, количество, категория, фирма и URL към изображение. Контролерът валидира входния

модел и при успех извиква услугата за продукти. Там първо се намира фирмата по име, след което се създава нов обект `Product`. Категорията се преобразува чрез `CategoryEnum.valueOf()`, а останалите полета се попълват директно от входния модел.

```
1 @Override
2 public void addProduct(
3     ProductAddBindingModel input,
4     Object email) throws Throwable {
5     Company company = companyRepository.findByName(input.getCompany())
6         .orElseThrow(() -> new Throwable("Company not found"));
7
8     Product product = new Product();
9     product.setUrl(input.getUrl());
10    product.setCategory(CategoryEnum.valueOf(input.getCategory()));
11    product.setCompany(company);
12    product.setPrice(input.getPrice());
13    product.setName(input.getName());
14    product.setQuantity(input.getQuantity());
15
16    productRepository.save(product);
17 }
```

Listing 3: Създаване на нов продукт в service слоя

Листинг 3 показва добре една типична service операция: извличане на зависим домейн обект, създаване на нов entity обект, попълване на полета и persistence чрез repository интерфейс. Решението е компактно и четимо, което е предимство за учебна разработка.

Каталог и филтриране по категории Каталогът е реализиран чрез маршрут от вида `/products/catalog/{category}`. В контролера се използва `switch` конструкция, която делегира към различни service методи според категорията. Тези service методи извличат всички продукти и филтрират резултатите по стойността на `CategoryEnum`. От архитектурна гледна точка това решение е достатъчно просто и разбираемо. То не е най-ефективният възможен подход, тъй като филтрирането се извършва на ниво Java поток върху колекцията от всички продукти, но за ограничен набор от данни е напълно приемливо.

Каталогът включва и клиентско филтриране по ценови диапазон, което използва REST крайната точка за извличане на максимална цена. Това е интересен пример за комбиниране на сървърно рендиране и допълнителна динамика от JavaScript.

Преглед на детайли и добавяне в количка За всеки продукт системата предоставя детайлен екран, от който потребителят може да заяви определено количество за добавяне в количката. Контролерът извиква метод за проверка на наличността, а при положителен резултат делегира към `service` логиката за реално добавяне на продукта.

В `ProductServiceImpl` това става чрез намаляване на наличността на продукта и добавяне или обновяване на запис в количката. Ако продуктът вече е намерен в количката, съществуващият запис се увеличава с новото количество; ако не съществува, създава се нов `ShoppingCartEntity`. Този модел е функционален, но тук се откроява и важна архитектурна слабост: проверката дали продуктът вече е в количката се извършва глобално върху всички записи в количката, а не само върху количката на текущия потребител. Освен това сравнението се базира на името на продукта, а не на комбинация от потребител и идентификатор на продукт. Това е достатъчно сериозно ограничение и следва да бъде изрично отбелязано.

3.1.5. Количка и финализиране на поръчка

Модулът за количка и checkout е реализиран чрез комбинация между `ProductController`, `ShoppingCartServiceImpl`, `UserServiceImpl` и `HistoryServiceImpl`. Това е една от най-интересните части на back-end логиката, защото координира няколко различни типа операции.

Преглед на количката При отваряне на количката контролерът добавя към модела списък от записи в количката, брой продукти, обща цена и крайна цена с доставка. Тук обаче отново е налице архитектурна особеност: методът `findAllShoppingCartEntities()` връща всички записи в количката, без филтриране по потребител. Това означава, че при реална многопотребителска среда визуализацията на количката не е строго изолирана на ниво потребител, което е съществено ограничение на текущата имплементация.

Въпреки това самият поток е ясно структуриран и лесен за проследяване. Потребителят вижда събраните продукти, цените и формата за адрес, а след това може да финализира поръчката чрез отделен POST маршрут.

Checkout логика и запис в историята Checkout процесът е реализиран в `HistoryServiceImpl` и е маркиран като `@Transactional`. Това е разумно решение, тъй като операцията променя множество състояния: създава нов исторически запис, обновява или

изчиства количката на потребителя, изтрива определени записи и премахва продукти с нулева наличност.

```
1 @Override
2 @Transactional
3 public void secureCheckout(
4     String address,
5     Object email,
6     Double totalPrice) throws Throwable {
7     User user = userRepository.findByEmail(email.toString())
8         .orElseThrow(() -> new Throwable("User not found"));
9
10    HistoryEntity historyEntity = new HistoryEntity();
11    historyEntity.setAddress(address);
12    historyEntity.setPrice(totalPrice);
13    historyEntity.setDate(LocalDate.now());
14    historyEntity.setUser(user);
15    historyRepository.save(historyEntity);
16
17    setNullToShoppingCartProducts(user.getId());
18    user.setProductsCart(new ArrayList<>());
19    user.getHistoryProducts().add(historyEntity);
20    userRepository.save(user);
21
22    shoppingCartRepository.deleteAllByUser_Id(user.getId());
23    deleteAllProductsWithZeroQuantity();
24 }
```

Listing 4: Същност на checkout операцията

Листинг 4 е особено показателен. От една страна, той демонстрира добре как service слой може да координира сложна домейн операция. От друга страна, разкрива и опростения характер на модела. Вместо реална поръчка с отделни позиции се създава обобщен HistoryEntity. Освен това при приключване на покупка продуктите с нулева наличност се изтриват напълно, вместо просто да останат в каталога като изчерпани. Това е валиден компромис за учебен проект, но в реална система би било по-удачно продуктите да останат видими с индикация за липса на наличност.

Изчистване на историята Историята може да бъде изчиствана от потребителя. Реализацията последователно премахва асоциациите от страна на потребителя, занулява връзката към потребителя в историческите записи и изтрива вече осиротелите записи. Тази логика демонстрира осъзнаване на релационните зависимости, макар и реализирано чрез опростен процедурен подход.

3.1.6. REST слой

Освен HTML ориентираното поведение системата включва и `MaxPriceRestController`, който обслужва маршрут `/api/products/max-price`. Контролерът извиква `ProductsService.getProductMaxPrice()` и връща `MaxPriceViewModel` чрез `ResponseEntity`. Реализацията е минималистична, но добре подбрана. Тя показва, че същият домейн модел може да бъде използван и за машинно четим интерфейс.

От гледна точка на учебната стойност това е важно, защото демонстрира как в едно и също приложение могат да съществуват както класически MVC маршрути, така и REST крайни точки. В последствие именно тази крайна точка се използва от JavaScript логиката в каталога за конфигуриране на клиентския ценови филтър.

3.1.7. Роля на repository интерфейсите в реализацията

Макар repository интерфейсите да са относително кратки, те играят важна роля в back-end имплементацията. Чрез тях услугите извличат потребители по имейл, фирми по име, продукти по идентификатор, записи от количка по потребител и история на поръчки по потребителски идентификатор. Този стил на работа подчертава предимството на Spring Data JPA: много от ежедневните persistence операции могат да бъдат описани с кратки и четими методи.

Същевременно именно на това ниво се вижда и една от възможностите за бъдеща оптимизация. Част от филтрирането в момента се извършва в service слоя след извличане на големи колекции. При по-натоварена система би било по-ефективно част от тази логика да бъде преместена към по-целенасочени repository заявки.

3.1.8. Наблюдавани силни страни на back-end реализацията

На база направения анализ могат да бъдат формулирани няколко положителни характеристики на back-end слоя:

- последователно използване на service слой за по-съществените бизнес операции;
- отделяне на HTML контролери и REST контролер;
- използване на mapper и binding модели за по-добра структурираност на данните;
- прилагане на password hashing и e-mail confirmation механизъм;

- използване на транзакционност при операция, която променя множество зависими състояния.

Тези качества правят системата убедителна като учебен пример и осигуряват добра основа за бъдещо надграждане.

3.1.9. Ограничения и технически рискове в back-end слоя

Наред със силните страни back-end кодът съдържа и няколко важни ограничения, които трябва да бъдат отчетени честно:

- контролът на достъпа е реализиран ръчно чрез сесийни проверки в контролерите, без централизирана security архитектура;
- checkout, количка и проверката за вече добавен продукт работят с логика, която не е строго изолирана по потребител;
- в част от операциите се разчита на твърдо зададени стойности, като URL за потвърждение и конфигурация за локално изпълнение;
- продуктите с нулева наличност се изтриват, вместо да преминават в състояние „изчерпан“;
- част от домейн логиката използва по-опростени структури, подходящи за учебен контекст, но недостатъчни за зряла търговска система.

Тези ограничения не следва да се разглеждат като провал на проекта. Напротив, те показват къде е естествената граница между функционален демонстрационен прототип и production-ready система.

3.2. Представящ слой и потребителско взаимодействие

Back-end логиката на една система за електронна търговия има стойност само ако бъде експонирана чрез последователен и използваем потребителски интерфейс. В разглеждания проект визуалният слой е реализиран чрез Thymeleaf шаблони, допълнени от CSS, Bootstrap базирани компоненти и ограничено количество JavaScript. Този избор е в пълно съответствие с общата архитектурна постановка на приложението: интерфейсът

е сървърно рендиран, което улеснява интеграцията между модели, валидация и HTML форми.

Настоящият раздел разглежда организацията на шаблоните, ролята на навигацията, локализацията, формите и основните UX решения. Анализът се базира върху реалните HTML шаблони в директорията `src/main/resources/templates` и статичните ресурси в `src/main/resources/static`.

3.2.1. Организация на шаблоните

Визуалният слой е структуриран около набор от отделни Thymeleaf шаблони, всеки от които отговаря на конкретен потребителски екран. Сред тях се открояват следните страници:

- `index.html` – начална страница и входна точка за неавтентикирани потребители;
- `login.html` и `register.html` – форми за удостоверяване и регистрация;
- `profile.html` – екран за преглед и редакция на профила;
- `add-company.html` – форма за добавяне на фирма;
- `add-product.html` – форма за добавяне на продукт;
- `products-catalog.html` – основен каталоген екран;
- `details.html` – детайлен изглед на конкретен продукт;
- `shopping-cart.html` – количка и форма за checkout;
- `history.html` – история на направените поръчки;
- `users.html`, `companies.html`, `delivery.html` и `error` шаблоните – допълнителни визуални компоненти на системата.

Този набор от шаблони е достатъчен, за да обхване целия потребителски цикъл в приложението. Важно е да се отбележи, че визуалният слой не е структуриран чрез централен layout механизъм или фрагменти, а по-скоро чрез отделни самостоятелни HTML файлове. Това прави проекта лесен за локално проследяване, но води и до значително дублиране на елементи като навигационната лента.

Таблица 6. Съответствие между шаблони и основни потребителски функции

Шаблон	Основна цел	Потребителски контекст
index.html	Представяне на начална страница и достъп до основни действия	Гост или неавтентикиран потребител
register.html	Създаване на нов акаунт	Гост
login.html	Вход в системата	Гост или непотвърден потребител
profile.html	Преглед и редакция на данни, списък с фирми	Удостоверен потребител
add-company.html	Въвеждане на фирма	Удостоверен потребител
add-product.html	Въвеждане на продукт	Удостоверен потребител
products-catalog.html	Каталоген преглед и филтриране	Удостоверен потребител
details.html	Избор на конкретен продукт и количество	Удостоверен потребител
shopping-cart.html	Обобщение на количка и checkout	Удостоверен потребител
history.html	Преглед и изчистване на историята	Удостоверен потребител

Таблица 6 показва, че интерфейсът е построен около ясни и тематично отделени екрани. Това е добра практика, защото всяка страница има разпознаваема функция и лесно може да бъде обвързана с конкретен контролерен маршрут.

3.2.2. Навигация и общи интерфейсни елементи

Един от най-видимите повторяеми елементи в шаблоните е навигационната лента. Тя присъства в почти всички екрани и включва връзки към регистрация, вход, профил, каталог, добавяне на продукт, история, количка, изход и смяна на езика. От гледна точка на потребителската логика това е силно решение: основните действия са достъпни от почти всеки екран и навигационната структура остава позната.

В същото време архитектурно решението има недостатък. Понеже навигацията се повтаря ръчно в множество шаблони, всяка бъдеща промяна трябва да бъде приложена на много места. При по-голямо приложение това би увеличило вероятността от несъответствия и би направило поддръжката по-трудна. По-зрял подход би бил използването на Thymeleaf fragments или общ layout шаблон. Независимо от това, за настоящия проект текущият вариант е лесен за проследяване и не затруднява разбирането на страниците.

Допълнителен общ елемент е визуалният фон, който в много страници използва фиксирано изображение и сходна цветова схема. Това придава визуална последователност, макар и реализирана чрез inline стилове, а не чрез централен CSS дизайн слой.

3.2.3. Thymeleaf като механизъм за свързване на данни и интерфейс

Thymeleaf [6] играе централна роля в представящия слой. Чрез атрибути като `th:text`, `th:field`, `th:each`, `th:if`, `th:href` и `th:action` шаблоните могат директно да използват данните, подготвени от контролерите. Това значително улеснява връзката между back-end слоя и HTML формите.

Особено полезно е използването на `th:field` при формите. По този начин се постига естествено свързване между binding модел и HTML полета, както и възможност за визуализиране на състояние при грешка. Например при формите за добавяне на фирма, продукт или редакция на профил, полетата се обвързват директно с модела, а условната визуализация на грешки използва `#fields.hasErrors(...)`.

```
1 <form th:action="@{/products/add-product}"
2     th:object="{productAddBindingModel}"
3     th:method="POST">
4   <label for="name">Name</label>
5   <input th:field="*{name}" id="name" type="text" class="form-control">
6   <p th:if="{#fields.hasErrors('name')}}" class="errors alert alert-danger">
7     Name must be between 5 and 20 characters.
```



```
8 </p>  
9 </form>
```

Listing 5: Типичен Thymeleaf фрагмент за форма

Листинг 5 показва как се реализира типичен формов сценарий. От една страна, HTML остава достатъчно четим. От друга страна, връзката с Java модела е непосредствена. Именно в това се състои едно от основните предимства на сървърно рендирания подход при Spring MVC приложения.

3.2.4. Форми и обработка на потребителски вход

Почти всички съществени действия в системата са реализирани чрез форми. Това включва регистрация, вход, редакция на профил, добавяне на фирма, добавяне на продукт, добавяне на количество в количката и checkout процеса. Формите са реализирани в стил, характерен за Bootstrap [15], с класове като `form-group`, `form-control`, `btn` и други визуални помощници.

В профилния екран се вижда сравнително богата композиция: форма за лични данни, визуализиране на текущите стойности и таблица с компаниите на потребителя. Това е добър пример за страница, която съчетава редакция и преглед на свързана информация. Формата за добавяне на фирма пък е по-компактна и логически разделя текстовите от числовите полета. Формата за добавяне на продукт е най-добре структурирана по отношение на домейнните данни, тъй като включва избор от `enum` категории и избор на фирма от предварително зареден списък.

От UX гледна точка позитивен аспект е, че при невалиден вход потребителят получава визуална обратна връзка чрез `alert` елементи. Това подобрява използваемостта и прави формовия поток по-ясен. Ограничение е, че съобщенията не са напълно стандартизирани на всички места, а част от тях са твърдо кодирани в самите HTML файлове.

3.2.5. Каталог и продуктови детайли

Каталогът е един от най-характерните екрани в системата. Той е реализиран като комбинация от навигационна зона, странично меню с категории, ценови филтър и галерия от продуктови карти. Визуалният модел се опира на Bootstrap, но включва и значително количество `inline CSS`, описващ оформлението на елементите, размерите на

картите, поведението на менюто и отзивчивостта на страницата.

Съществено UX качество на каталога е, че всеки продукт се визуализира с изображение, име, категория и цена, а бутонът за детайлен преглед отвежда потребителя към следващата стъпка от взаимодействието. В допълнение клиентският филтър по цена придава усещане за по-динамичен интерфейс, без да се изисква пълноценна SPA архитектура.

Страницата с детайли на продукта е по-концентрирана върху единичен обект. Тя представя изображение, име, цена и наличност, а формата за въвеждане на количество позволява директно добавяне към количката. От гледна точка на потребителския поток това е правилно решение: каталогът служи за избор, а страницата за детайли – за вземане на решение и заявяване на покупка.

3.2.6. Количка и checkout екран

Екранът за количка комбинира две взаимосвързани цели: визуализация на текущото съдържание и подготовка на финализирането на поръчката. Визуалната композиция разделя съдържанието на две части: списък с избраните продукти и summary панел със суми, доставка, адрес и бутон за checkout.

Това решение е потребителски удачно, тъй като показва едновременно детайла на избраните артикули и обобщената финансова рамка на поръчката. Формата за адрес е вградена директно в summary панела, което намалява броя на преходните екрани. От друга страна, самата страница е относително тежка като inline CSS и в production контекст би било по-удачно стиловете да се централизирани във външен ресурс.

3.2.7. Профил и история на поръчките

Профилният екран е сред най-богатите откъм комбинирано съдържание. Той събира както form-based редакция на основните потребителски полета, така и индиректно насърчава управлението на фирмите чрез страничен панел със списък от съществуващи компании и връзка за добавяне на нова. Това е добра UX практика, тъй като поставя логически свързаните операции в един и същ контекст.

Историята на поръчките е реализирана с тъмен табличен интерфейс, който се различава визуално от останалите екрани. Това прави страницата разпознаваема и внушава архивен характер на информацията. Таблицата показва идентификатор на поръчката, имейл, адрес, дата и цена. Допълнително е наличен бутон за изчистване

на историята. От потребителска гледна точка този екран изпълнява коректно ролята си, макар че в production система биха били желани филтриране, търсене и по-богат контекст за отделните поръчки.

3.2.8. Локализация и двуезичен интерфейс

Поддръжката на два езика е реализирана чрез `messages.properties` и `messages_bg.properties`, заедно с `LocaleChangeInterceptor`. В навигационната лента са добавени преки връзки `?lang=en` и `?lang=bg`, които позволяват смяна на езика чрез параметър в заявката. Това е прост, но ефективен модел за локализация в сървърно рендирана система.

От анализа на шаблоните се вижда, че голяма част от основните надписи използват message ключове чрез `#{...}`. Това е добро решение, защото държи езиковите ресурси извън самите шаблони. Съществуват обаче и частично локализирани или напълно твърдо кодирани елементи. Например част от текстовете за категории, някои помощни изрази и някои съобщения за грешка не са напълно синхронизирани между двата езика. Това означава, че локализацията е реално налична, но все още не е напълно завършена.

3.2.9. Използване на външни библиотеки и клиентски ресурси

Визуалният слой използва Bootstrap, Font Awesome, jQuery и jQuery UI [15, 16]. Това е видно както от шаблоните за каталог и детайли, така и от количеството CDN препратки в HTML файловете. От гледна точка на бързото разработване това е разбираем избор: готовите UI библиотеки позволяват по-бързо постигане на визуално приемлив резултат.

Наред с положителните страни тук има и няколко наблюдения. Първо, в различни шаблони се използват различни версии на Bootstrap. Второ, част от ресурсите са включени едновременно от локални файлове и от CDN. Трето, голямо количество стилове се дефинира inline във всеки отделен шаблон, вместо да се поддържа по-строга централизирана front-end организация. Това не е критичен проблем за малък проект, но е важен индикатор за бъдещи възможности за подобрене.

3.2.10. UX силни страни

Въпреки опростения си характер, интерфейсът има няколко съществени предимства:

- основните действия са видими и достъпни от навигацията;

- формите са ясно отделени по функции и показват грешки при входни проблеми;
- каталогът е визуално разпознаваем и поддържа филтриране по категория и цена;
- checkout екранът обединява продуктова информация и обобщение на поръчката;
- профилът и историята предлагат логично групирани потребителски сценарии.

Тези характеристики правят системата използвана и подходяща за демонстрация пред научен ръководител, комисия или крайни потребители в контекста на дипломната разработка.

3.2.11. Ограничения на представящия слой

Както и при back-end частта, представящият слой има своите ограничения:

- дублиране на навигационна логика и HTML блокове между шаблоните;
- смесване на външни CDN ресурси, локални файлове и inline CSS;
- частична, но не пълна локализация на интерфейса;
- отсъствие на централен layout или fragment система;
- наличие на твърдо кодирани визуални и текстови елементи, които намаляват гъвкавостта.

Тези ограничения обаче не отменят факта, че интерфейсът е функционален и достатъчно изразителен за целите на проекта. По-скоро те очертават естествена траектория за последващо професионализиране на front-end слоя.

3.3. Структура на проекта, конфигурация и експлоатационна среда

Освен архитектурата и домейн логиката, всяко завършено приложение следва да бъде разглеждано и от operational перспектива: как е организирано в хранилището, какви зависимости използва, как се компилира, къде се съхраняват ресурсите и какви артефакти се генерират в резултат от build процеса. За настоящата система това е особено важно, тъй като проектът използва класическа Maven организация, комбинирана със Spring Boot конфигурация, MySQL свързаност, SMTP настройка и набор от HTML ресурси.

Този раздел анализира именно този аспект на разработката и показва как логическата структура на приложението се материализира във файловата организация на проекта.

3.3.1. Обща организация на проекта

Проектът е структуриран като стандартно Maven приложение. В кореновата директория се намират `pom.xml`, Spring Boot wrapper файловете, директорията `src` със сорс код, директорията `target` с генерирани артефакти, както и директорията `doc`, в която е разположена настоящата дипломна документация. Това разделение е важно, защото позволява да се разграничат поне три различни аспекта на системата:

- **изходен код и ресурси** – реализирани в `src`;
- **резултат от компилация и копиране на ресурси** – реализирани в `target`;
- **академична документация** – реализирана в `doc`.

Тази подредба е полезна не само от практическа гледна точка, но и за дипломното описание, тъй като ясно отделя инженерния продукт от неговото документиране.

Таблица 7. Ключови директории в проекта и тяхното предназначение

Директория или файл	Предназначение
<code>pom.xml</code>	Описва зависимостите, версията на Java и build конфигурацията
<code>src/main/java</code>	Съдържа основния Java код на приложението
<code>src/main/resources/templates</code>	Съдържа Thymeleaf шаблоните
<code>src/main/resources/static</code>	Съдържа статични ресурси като CSS и JavaScript
<code>src/main/resources/application.properties</code>	Съдържа локалната конфигурация за база данни, mail и JPA
<code>target/classes</code>	Съдържа компилирани класове и копирани ресурси след build
<code>target/test-classes</code>	Съдържа тестови артефакти при наличие на компилирани тестове
<code>doc</code>	Съдържа LaTeX документацията и библиографията на проекта

Както се вижда от таблица 7, проектът следва добре позната и разбираема структура. Това е съществено предимство, защото разработчик, който е работил със Spring Boot и Maven, може бързо да се ориентира в него.

3.3.2. Описание на зависимостите в `pom.xml`

Файлът `pom.xml` е основният декларативен център на Java проекта. В него са зададени родителският Spring Boot артефакт, версията на Java и списъкът със зависимости. Анализът на файла показва, че системата използва сравнително типичен набор от библиотеки за Spring базирано уеб приложение.

Таблица 8. Основни зависимости и роля в проекта

Зависимост	Функция в приложението	Практическо значение
spring-boot-starter-web	Уеб инфраструктура и MVC маршрути	Осигурява основата на HTTP слоя [4, 5]
spring-boot-starter-thymeleaf	Сървърно рендиране HTML шаблони	Интегрира визуалния слой с Spring MVC [6]
spring-boot-starter-data-jpa	Persistence слой чрез JPA	Улеснява работата с релационната база [7]
mysql-connector-java	JDBC драйвер за MySQL	Позволява реална връзка с базата данни [17]
spring-boot-starter-validation	Валидация на входни данни	Използва се в binding моделите [10]
modelmapper	Преобразуване между модели	Намалява ръчния код при mapping [18]
spring-context-support и javax.mail	Поддръжка за e-mail интеграция	Използват се при потвърждение на регистрацията [19]
spring-security-crypto	Хеширане на пароли	Осигурява Pbkdf2Password Encoder [11]
spring-boot-starter-test	Тестова инфраструктура	Осигурява база за бъдещи тестове

Таблица 8 показва, че изборът на зависимости е в общия случай добре съобразен с целите на проекта. Той покрива всички ключови нужди: уеб слой, шаблони, persistence, валидация, mail интеграция и password hashing.

Въпреки това анализът на `pom.xml` разкрива и няколко важни наблюдения. На първо място, зависимостта `spring-security-crypto` присъства два пъти с различни версии. На второ място, `spring-boot-starter-data-jpa` е декларирана с версия `LATEST`, което не е добра практика в устойчив проект, защото може да доведе до по-трудно възпроизводими

build резултати. На трето място, някои security зависимости са коментирани, което подсказва, че проектът първоначално е бил разглеждан и като кандидат за по-богата security интеграция, но впоследствие е останал при по-опростен модел.

3.3.3. Версия на платформата и езика

Проектът използва Spring Boot 2.6.2 и Java 11. Това е разумна комбинация за учебно-приложна разработка. Java 11 е дългосрочно поддържана версия на езика и осигурява стабилна основа, а Spring Boot 2.6.2 предлага достатъчно зрял и добре документиран стек за уеб приложения. Тази комбинация гарантира съвместимост с използваните библиотеки и прави проекта лесен за стартиране в типична локална разработвателска среда.

3.3.4. Конфигурация на приложението

Операционното поведение на приложението се управлява основно от файла `src/main/resources/application.properties`. В него са зададени параметрите за логване, връзка към MySQL, JPA поведението, както и SMTP конфигурацията за електронната поща.

Конфигурацията на базата данни използва MySQL драйвер и URL към база `onlineShop`, като е включена опция за автоматично създаване на базата при нужда. Hibernate е настроен с `ddl-auto=update`, което означава, че схемата може да бъде синхронизирана с entity модела при стартиране. Това решение е удобно за локална разработка, защото намалява необходимостта от отделни миграционни скриптове в ранна фаза на проекта.

От operational гледна точка е важно и използването на `spring.jpa.open-in-view=false`. Това показва съзнателен избор да не се оставя persistence контекстът отворен през визуалния слой. Макар проектът да не използва сложни lazy loading сценарии в интерфейса, тази настройка е добра практика, защото намалява вероятността от неочаквани достъпи до базата по време на визуализация.

3.3.5. Конфигурация на електронната поща

Файлът `application.properties` съдържа и настройките за `spring.mail.host`, порт, SMTP автентикация, TLS и потребителско име. Тези параметри позволяват изпращане на потвърждаващ e-mail към новорегистрирани потребители. От функционална гледна точка това е значимо предимство, защото системата преминава отвъд чисто

локалните CRUD операции и демонстрира интеграция с външен инфраструктурен компонент.

Наред с това обаче тук се вижда и едно от най-важните operational ограничения на проекта: чувствителни данни като парола за e-mail акаунт и данни за достъп до базата са записани директно в конфигурационен файл. Това е приемливо за локална учебна среда, но не следва да бъде допускано в production режим. По-зрял подход би включвал използване на environment variables, секретно хранилище или отделни профили за различни среди.

3.3.6. Build процес и Maven жизнен цикъл

Maven [20] осигурява стандартизиран жизнен цикъл за компилация, тестване и пакетиране. Дори когато проектът не използва сложни custom plugin сценарии, самото наличие на Maven build носи няколко преимущества:

- централизирано управление на зависимостите;
- възпроизводим build процес на различни машини;
- ясна конвенция за разположение на сорс код и ресурси;
- лесна интеграция със Spring Boot Maven plugin за стартиране и пакетиране.

В рамките на настоящия проект build процесът води до попълване на директорията target, където се намират classes, test-classes, както и стандартни поддиректории за генерирани сорсове. Най-важната от тях е target/classes, в която се копират както компилираните .class файлове, така и ресурсите от src/main/resources, включително HTML шаблоните.

3.3.7. Съдържание и роля на target

От наличната структура се вижда, че target съдържа поне следните поддиректории: classes, generated-sources, generated-test-sources и test-classes. Това е очакван резултат от Maven build процес. Практическото значение на тази директория е двойно.

Първо, тя съдържа реално изпълнимите артефакти на приложението след компилация. Второ, тя предоставя добра възможност за проверка дали ресурсите от templates и static са коректно копирани и достъпни за изпълнимата среда. В конкретния

проект например шаблоните от `src/main/resources/templates` са налични и в `target/classes/templates`, което потвърждава правилното участие на ресурсите в build процеса.

3.3.8. Практическа среда за изпълнение

За локално стартиране на приложението са необходими поне следните предпоставки:

- налична Java 11 среда;
- работещ MySQL сървър;
- валидни данни за достъп до базата, съобразени с `application.properties`;
- достъпна e-mail конфигурация, ако се тества регистрационният поток;
- Maven или използване на включения wrapper за стартиране на проекта.

Тази operational рамка е характерна за Spring Boot приложения и не изисква специализирана инфраструктура. Именно това е още едно предимство на избрания стек: проектът може да бъде разработван и демонстриран в сравнително стандартна среда без значителни допълнителни инструменти.

3.3.9. Операционни рискове и възможности за подобрене

Анализът на конфигурацията позволява да бъдат формулирани няколко operational риска:

- чувствителни параметри са съхранени в изходния код вместо във външна конфигурация;
- използването на LATEST в зависимост създава риск за повторемостта на build процеса;
- наличието на дублирани зависимости подсказва необходимост от по-строга dependency хигиена;
- липсват ясно разграничени профили за development и production среда;
- отсъстват миграции на схемата чрез инструмент като Flyway или Liquibase.

Въпреки това operational картината на системата остава напълно достатъчна за целите на дипломната разработка. Проектът е стартиращ, конфигурируем и използва стандартни Java инструменти за компилация и изпълнение. Това го прави едновременно практичен и методологично удобен за анализ.

3.3.10. Обобщение на програмната реализация

Програмната реализация на проекта демонстрира ясно приложени принципи на Spring базирана уеб система: контролери за управление на HTTP потока, service слой за домейн логика, repository интерфейси за persistence, mapper за трансформации и инфраструктурна конфигурация за повторно използвани компоненти. Представящият слой допълва тази организация чрез Thymeleaf шаблони и сървърно рендирани форми. Структурата на проекта, Maven конфигурацията и operational средата осигуряват добра основа за локално стартиране и анализ.

Същевременно направеният преглед показва и конкретни области за подобрене – най-вече по отношение на сигурността, изолацията на потребителските данни, централизацията на front-end слоя и управлението на чувствителна конфигурация. Точно тази комбинация от работеща функционалност и ясно видими точки за развитие придава на проекта неговата инженерна стойност. Следващата глава представя ръководство за използване на системата чрез поредица потребителски сценарии и верификация на функционалността.

4. РЪКОВОДСТВО ЗА ИЗПОЛЗВАНЕ

Тази глава представя ръководство за използване на системата чрез поредица потребителски сценарии, последвани от верификация на реализираната функционалност. Първо се проследяват основните демонстрационни сценарии, които обхващат целия жизнен цикъл на работа със системата. След това се представят използваният подход към верификацията и матрицата за функционална верификация, които потвърждават, че описаните сценарии имат реална програмна основа.

4.1. Потребителски сценарии

Една от важните функции на техническата дипломна документация е да направи връзка между архитектурното описание и реалната употреба на системата. Поради тази причина в настоящия раздел са представени основните потребителски сценарии в последователен демонстрационен вид. Всеки сценарий е придружен от описание на очакваното поведение на системата и от предварително предвидено място за screenshot от работещото приложение.

Този подход има две предимства. От една страна, текстът действа като кратко ръководство за демонстрация. От друга страна, структурата позволява реални екранни снимки да бъдат добавени на по-късен етап без нужда от пренаписване на документа. Така дипломната работа остава едновременно съдържателно завършена и practically подготвена за визуално допълване.

4.1.1. Сценарий 1: Начална страница и избор на език

Демонстрацията започва от началната страница на приложението. Тя има ролята на публична входна точка и предоставя достъп до регистрация, вход и избор на език. При липса на активна сесия системата не позволява използване на вътрешните функции, а се държи като портал към удостоверителния процес. Визуално началната страница съдържа хедър, навигационна лента, hero секция и основни бутони за навигация.

От потребителска гледна точка това е правилен първи екран, защото поставя акцент върху най-важните начални действия: регистриране или вход. Паралелно с това наличието на връзки за EN и BG в навигацията демонстрира, че приложението поддържа базова локализация още от първия момент на взаимодействие.

Placeholder за screenshot

Начална страница и езиков превключвател

Тук да се постави screenshot на началната страница, на който ясно да се виждат навигационната лента, hero секцията и връзките за смяна на езика.

Фигура 4. Начална страница и езиков превключвател

4.1.2. Сценарий 2: Регистрация на нов потребител

Следващата естествена стъпка е регистрацията. Потребителят отваря формата за регистрация и попълва име, фамилия, имейл и парола. При коректно подадени данни системата създава нов профил и инициира изпращане на потвърждаващ e-mail. Ако входът не е коректен или ако вече съществува потребител със същия имейл, се визуализира подходяща обратна връзка.

Този сценарий има ключова демонстрационна стойност, защото показва едновременно работа с форма, server-side обработка, persistence операция и интеграция с външен mail компонент. Именно тук се вижда, че проектът надхвърля чисто визуален прототип и реализира реален поток по създаване на акаунт.

Placeholder за screenshot

Форма за регистрация

Тук да се постави screenshot на екрана за регистрация с попълнени примерни данни или с демонстрация на валидационни съобщения при невалиден вход.

Фигура 5. Форма за регистрация

4.1.3. Сценарий 3: Потвърждение на акаунт и вход

След изпращане на регистрационната форма потребителят следва да потвърди своя акаунт чрез връзката, получена по електронна поща. В практическа демонстрация може да се покаже както самият имейл, така и резултатът от активиране на линка – преминаване към форма за вход. След потвърждение потребителят въвежда имейл и парола, а системата проверява валидността на данните и създава активна сесия.

От гледна точка на потребителската история това е преходът от публичната към вътрешната част на приложението. От този момент нататък системата започва да показва допълнителни навигационни елементи, които са достъпни само при активна сесия.

Placeholder за screenshot

Вход в системата след потвърдена регистрация

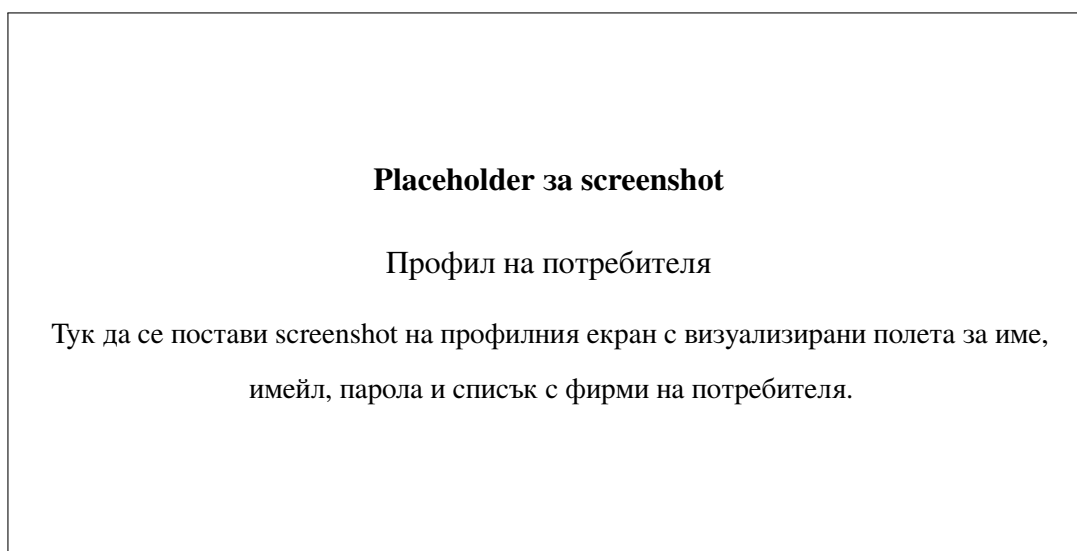
Тук да се постави screenshot на login формата или на резултата след успешно удостоверяване, например пренасочване към каталога.

Фигура 6. Вход в системата след потвърдена регистрация

4.1.4. Сценарий 4: Профил на потребителя и списък с фирми

След вход един от най-важните екрани е профилът. Той позволява на потребителя да прегледа своите основни данни и при необходимост да ги редактира. Допълнително в страничната част на екрана се визуализира списък от свързани фирми и е наличен бутон за добавяне на нова.

Този екран е показателен за комбиниран характер на интерфейса: той не е само форма за редакция, а едновременно работна зона, която обединява идентичност на потребителя и свързана бизнес информация. При демонстрация е подходящо да се покаже както нормалното състояние на профила, така и пример за съобщение при неправилно въведена стара парола или несъвпадение между новата и потвърдената парола.



Фигура 7. Профил на потребителя

4.1.5. Сценарий 5: Добавяне на фирма

Формата за добавяне на фирма е логически подготвителна стъпка за последващото въвеждане на продукти. Потребителят посочва име на фирмата, идентификационни данни, адрес и тип на регистрация. След успешен запис фирмата става достъпна в списъка от фирми и може да бъде избрана от формата за нов продукт.

Този сценарий е особено полезен за демонстрация пред комисия, защото показва, че приложението не работи с изолирани продукти, а поддържа по-реалистична домейн структура, в която продуктът се асоциира с фирма. По този начин каталогът се обвързва с източник на данни, а не с анонимни артикули.

Placeholder за screenshot

Форма за добавяне на фирма

Тук да се постави screenshot на екрана за добавяне на фирма, включително полетата за име, MOL, VAT, адрес и статус на регистрация.

Фигура 8. Форма за добавяне на фирма

4.1.6. Сценарий 6: Добавяне на продукт

След като съществува поне една фирма, потребителят може да премине към добавяне на продукт. Екранът за това включва полета за име, URL към изображение, цена, количество, категория и избор на фирма. Категорията се избира от предварително дефиниран enum списък, а фирмата – от асоциираните с потребителя компании.

Този сценарий показва няколко важни характеристики на приложението. Първо, демонстрира комбиниране на текстови, числови и избираеми полета. Второ, показва как входните данни се валидират още на ниво форма. Трето, илюстрира как домейн връзките вече се материализират в UI: потребителят не въвежда фирма на свободен текст, а я избира от наличен набор.

Placeholder за screenshot

Добавяне на нов продукт

Тук да се постави screenshot на формата за добавяне на продукт, на който ясно да се виждат изборът на категория, фирма и валидационните полета.

Фигура 9. Добавяне на нов продукт

4.1.7. Сценарий 7: Каталог и филтриране по категория и цена

След добавяне на продукти системата може да бъде демонстрирана чрез каталога. Това е един от най-визуалните екрани в приложението. Потребителят вижда странично меню с категории и зона с продуктови карти. Допълнително JavaScript логиката използва REST крайната точка за максимална цена, за да конфигурира ценови плъзгач за клиентско филтриране.

При демонстрация е подходящо да се покажат поне три действия: отваряне на общия каталог, преминаване към конкретна категория и стесняване на видимите продукти чрез ценовия диапазон. Този сценарий представя най-добре съчетанието между сървърно рендиран интерфейс и ограничена клиентска динамика.

Placeholder за screenshot

Каталог с продукти

Тук да се постави screenshot на каталога с няколко продукта, страничното меню с категории и активния ценови филтър.

Фигура 10. Каталог с продукти

4.1.8. Сценарий 8: Детайли на продукт и добавяне в количка

След избор на конкретен артикул потребителят преминава към страницата с детайли. Там се визуализират изображение, наименование, цена и текуща наличност. Потребителят въвежда желаното количество и го добавя в количката. Ако количеството е невалидно или надвишава наличността, системата връща съобщение за грешка.

Този екран има съществена демонстрационна роля, защото именно тук се вижда преходът от пасивно разглеждане към активна транзакционна интеракция. От UX гледна точка формата е минималистична и фокусира вниманието върху единичния продукт.

Placeholder за screenshot

Детайлен изглед на продукт

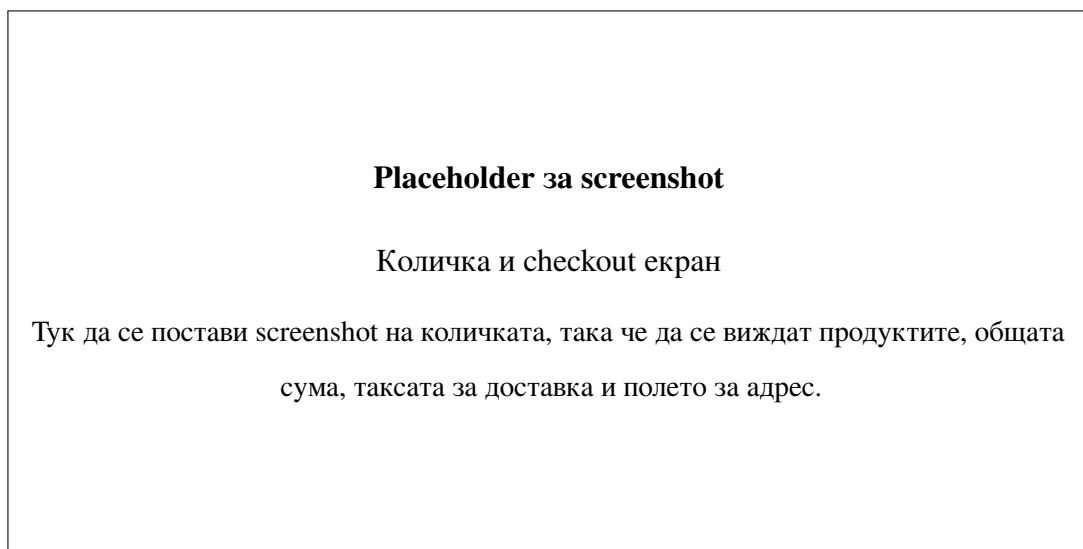
Тук да се постави screenshot на страницата с детайли на продукт, включително полето за количество и бутона за добавяне в количката.

Фигура 11. Детайлен изглед на продукт

4.1.9. Сценарий 9: Количка и финализиране на поръчка

След добавяне на продукти в количката потребителят отваря съответния екран, където вижда избраните артикули, броя им, цената на всеки запис, общата стойност, таксата за доставка и крайна сума. В summary панела се попълва адресът, след което се извършва checkout.

Този сценарий е особено важен, защото демонстрира, че приложението не спира до каталог и визуализация, а реализира реален край на потребителската цел. При защита на дипломна работа именно тази страница често е сред най-силните екрани за демонстрация, защото интегрира back-end изчисления, визуално обобщение и form-based завършване на операцията.



Фигура 12. Количка и checkout екран

4.1.10. Сценарий 10: Екран за успешна поръчка и история

След завършване на покупката потребителят се пренасочва към потвърждаващ екран, а впоследствие може да отвори своята история на поръчките. Историческият изглед показва списък с реализираните покупки, включително идентификатор, имейл, адрес, дата и цена. Наличен е и бутон за изчистване на историята.

Този етап е важен за демонстрацията, защото визуализира персистентния резултат от целия потребителски процес. Докато каталожните и checkout екраните показват текущо състояние, историята показва, че системата поддържа и времева следа за вече реализирани действия.

Placeholder за screenshot

История на поръчките

Тук да се постави screenshot на екрана с история на поръчките, включително таблицата с дата, адрес и цена на направените покупки.

Фигура 13. История на поръчките

4.1.11. Сценарий 11: Демонстрация на локализацията

Допълнителен, но важен демонстрационен сценарий е смяната на езика. Тъй като системата поддържа базова английска и българска локализация, е полезно в дипломната работа да се покаже поне един и същи екран в двата режима. Това дава визуално доказателство, че локализационната конфигурация не е само декларативно налична, а реално участва в поведението на приложението.

При добра демонстрация могат да бъдат сравнени например началната страница, формата за добавяне на продукт или екранът за профил в двата езикови режима. Подобна съпоставка е особено ценна, защото показва и ограниченията на текущата локализация – къде тя е пълна и къде остават твърдо кодирани текстове.

Placeholder за screenshot

Съпоставка между английски и български интерфейс

Тук да се постави двойка screenshots или комбиниран кадър, показващ един и същ екран на английски и на български език.

Фигура 14. Съпоставка между английски и български интерфейс

4.1.12. Препоръки за създаване на screenshots

За да бъде визуалната част на дипломната работа достатъчно качествена, е препоръчително screenshots да бъдат направени при следните условия:

- браузърът да бъде отворен на достатъчно голяма резолюция, така че всички основни елементи да се виждат ясно;
- демонстрационните данни да бъдат подготвени предварително, за да няма празни списъци или непълни форми;
- при нужда да се подберат данни с по-описателни имена на фирми и продукти;
- да се избягва показване на чувствителни конфигурационни данни, локални пароли и реални секрети;
- да се поддържа последователен визуален стил между всички използвани изображения.

Тези препоръки са важни, защото screenshots не са просто украшение към документа. Те са доказателствен материал, че описаните сценарии са реално демонстрируеми и че системата има работещ потребителски интерфейс.

4.2. Верификация на функционалността

След представянето на потребителските сценарии следва да се покаже, че те имат реална програмна опора в кода на приложението. Целта на този раздел не е просто

да се повтори, че системата съдържа дадена функционалност, а да се покаже доколко решението удовлетворява поставените изисквания и какви са доказателствата за неговото съществуване.

В настоящия случай верификацията е извършена предимно чрез аналитичен подход върху реалния код, маршрути, шаблони, конфигурация и взаимовръзки между модулите. Това означава, че проверката се базира на действително реализираната логика на приложението, а не само на намерения или проектни декларации. Тъй като проектът има учебно-приложен характер и липсва богата автоматизирана тестова база, именно този тип инженерно-аналитична верификация е най-подходящ за дипломната работа.

4.2.1. Подход към верификацията

За целите на настоящата верификация са използвани няколко взаимнодопълващи се подхода.

Статичен анализ на сорс кода Първият подход е проследяване на действителната логика в контролерите, услугите, repository интерфейсите, entity моделите и шаблоните. Чрез него може да бъде установено дали отделните потребителски сценарии имат реална програмна опора, каква е последователността на операциите и къде са разположени ключовите проверки за валидност, достъп или коректност на данните.

Сценарийна проверка на функционалността Вторият подход се базира на дефинираните по-горе потребителски сценарии. За всеки от тях се проверява дали в системата има наличен контролерен маршрут, service логика, подходящ шаблон и persistence поведение, които заедно реализират целевия резултат. По този начин се формира сценарийна матрица на верификация.

Архитектурна и качествена оценка Третият подход е по-качествен и разглежда не само наличието на дадена функция, а начина, по който тя е реализирана. Тук се оценяват свойства като структурираност на кода, повторна използваемост, разширяемост, защита на чувствителни данни, consistency на интерфейса и operational дисциплина.

Съвкупността от тези три перспективи дава достатъчно убедителна рамка за верификация на проекта.

4.2.2. Матрица за функционална верификация

В таблица 9 е представена обобщена матрица за функционална верификация. Тя не замества автоматизираното тестване, но систематизира дали ключовите функции са реално представени в кода и какви са доказателствата за тяхното съществуване.

Таблица 9. Матрица за функционална верификация

Сценарий	Проверявана функционалност	Основа за верификация	Извод
V-1	Регистрация на потребител	Налични <code>/users/register</code> GET/POST маршрути, <code>service</code> логика за запис и <code>mail</code> изпращане	Функцията е реализирана последователно и включва <code>persistence</code> и <code>e-mail</code> стъпка
V-2	Потвърждение на акаунт	Наличен маршрут <code>/users/{userId}/confirm-email</code> и <code>service</code> метод <code>confirm</code>	Потвърждението е реализирано в опростен, но работещ вид
V-3	Вход и изход	Налични <code>login</code> маршрути, проверка на парола и <code>invalidate</code> на сесията	Функцията е налице, но е базирана на ръчно сесийно управление
V-4	Редакция на профил	Налични <code>profile</code> маршрути, проверки на стара парола и <code>matching</code> на нови пароли	Функцията е реализирана с базова входна защита
V-5	Добавяне на фирма	<code>CompanyController</code> , <code>AddCompanyBindingModel</code> , <code>CompanyServiceImpl</code>	Реализацията е ясна и логически свързана с потребителя
V-6	Добавяне на продукт	<code>ProductController</code> , <code>ProductAddBindingModel</code> , <code>ProductServiceImpl</code>	Налични са всички необходими елементи за запис на продукт
V-7	Каталог по категории	Категорийни маршрути, шаблон <code>products-catalog.html</code> , <code>enum</code> категории	Функцията е реализирана, макар и с филтриране на ниво <code>service</code> вместо <code>repository</code>

Сценарий	Проверявана функционалност	Основа за верификация	Извод
V-8	Детайлен преглед и добавяне в количка	Маршрут <code>/products/{id}/details</code> , проверка на количество, <code>update</code> на количката	Реализацията е налична, но изолацията по потребител е непълна
V-9	Checkout	<code>shopping-cart</code> POST маршрут и <code>HistoryServiceImpl.secureCheckout</code>	Функцията е реализирана чрез транзакционна координация на няколко операции
V-10	История на поръчките	<code>/products/my-history</code> GET/POST маршрути и <code>HistoryRepository</code>	Налице е основен исторически модул с възможност за изчистване
V-11	Локализация	<code>messages.properties</code> , <code>messages_bg.properties</code> , <code>I18nConfig</code>	Поддръжката е реална, но частично непълна
V-12	REST крайна точка	<code>MaxPriceRestController</code> и <code>service</code> метод за максимална цена	Налице е примерна API функционалност

Матрицата показва, че всички основни функции, формулирани в изискванията, имат конкретни програмни реализации. Това е съществен резултат, защото доказва, че системата не е само архитектурно упражнение, а реален функционален софтуерен продукт. В същото време няколко от сценариите ясно подсказват и ограничителни фактори, които ще бъдат разгледани в Заключение.

ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа разглежда последователно проектирането и разработването на уеб базирана система за електронна търговия с компютърна и офис техника. В рамките на разработката беше създаден реален Java Spring Boot проект, който покрива основните потребителски и бизнес процеси на един компактен онлайн магазин: регистрация, потвърждение на акаунт, вход, поддръжка на профил, управление на фирми, добавяне на продукти, разглеждане на каталог, работа с количка, финализиране на поръчка и поддържане на история на покупки. Една от основните цели на труда беше не просто да се реализира работещо приложение, а да се покаже как подобна система може да бъде структурирана по ясен, анализируем и методологично последователен начин. В този смисъл разработката демонстрира важна инженерна стойност: архитектурата е организирана в отделни слоеве и пакети, домейн моделът е достатъчно ясен и свързан с предметната област, а избраният технологичен стек е едновременно практичен и представителен за съвременна Java уеб разработка.

Проведеният анализ показва, че изборът на Spring Boot, Spring MVC, Thymeleaf, Spring Data JPA и MySQL е напълно оправдан за подобен тип задача. Тези технологии позволяват бързо, но дисциплинирано изграждане на уеб приложение с добра проследимост на логиката от потребителския интерфейс до persistence слой. Използването на Hibernate Validator, ModelMapper, JavaMail интеграция и Pbkdf2PasswordEncoder допълнително повишава качеството на решението и демонстрира осъзнато използване на наличните средства в екосистемата. От operational гледна точка обаче dependency конфигурацията би могла да бъде по-чиста, особено по отношение на дублираните или твърде свободно декларираните версии, което е една от посоките за бъдеща хигиенизация.

От гледна точка на качествените характеристики проектът се представя на добро ниво за своя мащаб. Архитектурната подреденост е сред най-силните му страни – налице е ясно разграничение между контролери, service слой, repository интерфейси, entity модели и визуален слой, а използването на отделни binding, service и view модели е свидетелство за осъзнато проектиране, а не за ad hoc имплементация. Поддържаемостта също е на добро ниво: кодът е сравнително четим, класовете имат разпознаваеми роли, а пакетната структура е логична, което позволява на нов разработчик бързо да се ориентира в системата. Използваемостта на приложението е приемлива и дори добра за учебен проект –

основните функции са групирани логично, навигацията е ясно видима и потребителят може да премине през целия жизнен цикъл на покупката без да губи контекст. По отношение на сигурността системата демонстрира няколко правилни решения като хеширане на паролите и забрана за вход на непотвърден потребител, но достъпът се управлява чрез `HttpSession` и ръчни условни проверки без централизирана security рамка, което я прави достатъчна за учебно-приложна демонстрация, но не и за production внедряване без значително доработване. Разширяемостта е добра, защото добавянето на нова страница следва установения модел: нов контролерен маршрут, нов шаблон, при нужда нов service метод и repository взаимодействие.

Особено значение в рамките на работата има фактът, че проектът не беше разгледан само описателно, а критично. Наред с положителните характеристики бяха идентифицирани и конкретни ограничения, които очертават естествената граница между функционален демонстрационен прототип и production-ready система. Най-значимото от тях е свързано със сигурността: макар паролите да се съхраняват чрез `Pbkdf2PasswordEncoder`, достъпът до защитените страници се контролира предимно чрез ръчни проверки върху `HttpSession`, а липсват роля-базирана авторизация, security filter chain и формален security context. Втората важна граница е домейнното моделиране на поръчките – текущата реализация използва `HistoryEntity` като обобщение на завършена покупка, но не поддържа отделни позиции, статуси, платежни състояния или жизнен цикъл на поръчката. Същевременно част от логиката за количката и продуктите работи върху глобални колекции, вместо да бъде строго филтрирана по текущия потребител, което в реална многопотребителска среда би довело до логически несъответствия. Наличието на credentials в `application.properties` е operational компромис, който макар да не пречи на демонстрационното стартиране, ограничава възможността проектът да бъде безопасно споделян или разгръщан без допълнителна подготовка. Front-end организацията остава сравнително фрагментирана с повторение на навигационна лента в множество шаблони, inline CSS и различни версии на външни библиотеки, а локализацията е само частично последователна. Накрая, макар проектът да използва стандартната Maven структура с тестова зависимост, липсва развита стратегия за unit, integration или UI тестове, така че качеството на системата в момента зависи в по-голяма степен от ръчна проверка и кодов анализ.

Тези наблюдения са важни, защото показват, че разработката е осмислена не

само като демонстрационен резултат, но и като основа за по-нататъшно професионално развитие. Най-естествената следваща стъпка е пълноценно въвеждане на Spring Security, което би позволило централизиран контрол върху достъпа до маршрути, разграничаване на роли като администратор, оператор и обикновен клиент, използване на стандартизиран authentication flow и по-лесно управление на сесии, logout и защита от неоторизиран достъп. Втората високоприоритетна посока е преработване на модела за количка и поръчки чрез въвеждане на отделни класове от тип Order и OrderItem и по-ясно разграничение между активна количка и историческа поръчка, така че системата да поддържа множество позиции в рамките на една поръчка, статуси, запазване на артикулите в историята без загуба на детайлна информация и по-прецизна отчетност. Operational зрелостта на проекта може значително да се подобри чрез използване на профили за development, test и production среда, изнасяне на чувствителните данни в environment variables или секретно хранилище и въвеждане на миграции на схемата чрез инструмент като Flyway или Liquibase. Front-end слой би се възползвал от извличане на повторяемите навигационни блокове в Thymeleaf fragments, централизиране на inline CSS във външни стилкови файлове, уеднаквяване на използваните версии на Bootstrap, разширяване на локализацията и подобряване на responsive поведението. Накрая, въвеждането на разумна тестова стратегия с unit тестове за service логиката, integration тестове за repository и контролери и сценарийни тестове за регистрация, вход, checkout и история би повишило увереността при бъдещи промени.

Освен приоритетните подобрения съществуват и по-широки направления за развитие, които биха трансформирали системата от учебен прототип към по-зряло приложение: разширяване на каталога с търсене, сортиране, странициране, маркиране на любими продукти и реално качване на изображения; интеграция с платежни оператори, куриерски услуги и по-богат REST API за външни клиенти; както и контейнеризация с Docker и автоматизирано разгръщане чрез CI/CD процес, които биха улеснили повторяемото стартиране на приложението. За по-голяма яснота тези насоки могат да бъдат обобщени в кратка приоритетна пътна карта, представена в таблица 10.

Таблица 10. Предложена пътна карта за бъдещо развитие

Приоритет	Направление	Очакван ефект
Висок	Spring Security и роли	По-сигурен и по-чист модел на достъп
Висок	Рефакториране на количка и поръчки	Коректност в многопотребителска среда и по-богат домейн модел
Висок	Изнасяне на secret конфигурация	По-добра operational безопасност
Среден	Централизация на front-end и локализация	По-добра поддръжка и UX последователност
Среден	Автоматизирани тестове	По-висока надеждност при развитие на кода
Среден	Миграции и профили за среди	По-предсказуем build и deployment процес
Нисък към среден	Разширени търговски функции	По-голяма практическа стойност на приложението
Нисък към среден	Контейнеризация и CI/CD	По-лесно разгръщане и възпроизводимост

Развитието на проекта може да бъде планирано стъпково: най-напред следва да се адресират качествата, които пряко влияят на коректността и сигурността, а след това да се премине към по-богати функционални и operational подобрения. От академична гледна точка основните приноси на настоящата работа могат да бъдат обобщени в няколко направления – систематизирано е реално приложение за електронна търговия в последователна многослойна архитектурна рамка, изведен е ясен модел на предметната област, демонстрирано е как чрез Spring технологии може да бъде реализиран пълен потребителски поток в уеб приложение, извършена е аналитична верификация на функционалността, интерфейса, конфигурацията и operational средата, и са формулирани реалистични насоки за бъдещо развитие към по-сигурна и по-зряла система. Тези приноси

потвърждават, че дипломната работа изпълнява своята двойна задача: от една страна представлява практическа инженерна реализация, а от друга – завършен технически документ, който обосновава използваните решения. Това е особено важно в контекста на специалност като компютърно и софтуерно инженерство, където способността да се премине от идея към добре структурирана реализация и след това към аргументирана документация е съществен показател за професионална зрялост.

Разработеното приложение не следва да бъде интерпретирано като крайна production-ready търговска платформа – неговата реална стойност е в това, че представлява работещ, концептуално ясен и надграждаем прототип, върху който могат да бъдат наслагвани следващи итерации с по-силна сигурност, по-богат order management модел, по-чист front-end слой, по-добра конфигурационна дисциплина и автоматизирано тестване. Точно тази отвореност към развитие е един от най-ценните резултати на проекта. В заключение може да се обобщи, че поставената цел е постигната: създадена е уеб базирана система за електронна търговия с компютърна и офис техника, която покрива ключов набор от функционалности, реализирана е с подходящи съвременни технологии, описана е в академично издържана форма и е оценена както през призмата на постигнатото, така и през призмата на нейните ограничения. Това прави проекта убедителен завършек на дипломна разработка и стабилна основа за бъдещо инженерно надграждане.

ИЗПОЛЗВАНА ЛИТЕРАТУРА

- [1] Shopify Inc., “Shopify help center and merchant documentation.” <https://help.shopify.com/en/manual>, 2026. Official documentation, accessed 2026-04-07.
- [2] Automattic, “Woocommerce documentation.” <https://woocommerce.com/documentation/>, 2026. Official documentation, accessed 2026-04-07.
- [3] Adobe Inc., “Adobe commerce and magento open source documentation.” <https://developer.adobe.com/commerce/docs/>, 2026. Official documentation, accessed 2026-04-07.
- [4] VMware, Inc., “Spring boot reference documentation, version 2.6.2.” <https://docs.spring.io/spring-boot/docs/2.6.2/reference/htmlsingle/>, 2026. Official documentation, accessed 2026-04-07.
- [5] VMware, Inc., “Spring framework reference documentation: Web mvc.” <https://docs.spring.io/spring-framework/docs/5.3.15/reference/html/web.html>, 2026. Official documentation, accessed 2026-04-07.
- [6] Thymeleaf Team, “Using thymeleaf.” <https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html>, 2026. Official documentation, accessed 2026-04-07.
- [7] VMware, Inc., “Spring data jpa reference documentation.” <https://docs.spring.io/spring-data/jpa/docs/2.6.0/reference/html/>, 2026. Official documentation, accessed 2026-04-07.
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [9] I. Sommerville, *Software Engineering*. Pearson, 10th ed., 2015.
- [10] Hibernate.org, “Hibernate validator reference guide.” https://docs.jboss.org/hibernate/validator/reference/en-US/html_single/, 2026. Official documentation, accessed 2026-04-07.
- [11] VMware, Inc., “Spring security reference: Password storage.” <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>, 2026. Official documentation, accessed 2026-04-07.

- [12] OWASP Foundation, “Password storage cheat sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026. Security guidance, accessed 2026-04-07.
- [13] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [14] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [15] Bootstrap Authors, “Bootstrap documentation, version 5.1.” <https://getbootstrap.com/docs/5.1/getting-started/introduction/>, 2026. Official documentation, accessed 2026-04-07.
- [16] OpenJS Foundation, “jQuery api documentation.” <https://api.jquery.com/>, 2026. Official documentation, accessed 2026-04-07.
- [17] Oracle Corporation, “Mysql 8.0 reference manual.” <https://dev.mysql.com/doc/refman/8.0/en/>, 2026. Official documentation, accessed 2026-04-07.
- [18] ModelMapper Contributors, “Modelmapper user manual.” <https://modelmapper.org/user-manual/>, 2026. Official documentation, accessed 2026-04-07.
- [19] VMware, Inc., “Spring framework reference documentation: Email integration.” <https://docs.spring.io/spring-framework/reference/integration/email.html>, 2026. Official documentation, accessed 2026-04-07.
- [20] Apache Software Foundation, “Apache maven project documentation.” <https://maven.apache.org/guides/>, 2026. Official documentation, accessed 2026-04-07.